Robust Navigation in an Unknown Environment With Minimal Sensing and Representation

Fulvio Mastrogiovanni, *Student Member, IEEE*, Antonio Sgorbissa, *Member, IEEE*, and Renato Zaccaria, *Member, IEEE*

***Abstract*—This paper presents μNav, a novel approach to navigation which, with minimal requirements in terms of onboard sensory, memory, and computational power, exhibits way-finding behaviors in very complex environments. The algorithm is intrinsically robust, since it does not require any internal geometrical representation or self-localization capabilities. Experimental results, performed with both simulated and real robots, validate the proposed theoretical approach.**

***Index Terms*—Bug algorithms, minimal sensing, navigation.**

I. Introduction

RESEARCH in mobile robot navigation is expected to satisfy two main design requirements.

1) *Completeness* assures that a path to a goal location can be found whenever it exists.
2) *Robustness* guarantees that the planned trajectory can be performed efficiently in case of incomplete or unreliable knowledge and sensor noise.

During the past few years, the first issue has been exhaustively investigated; many complete algorithms (in a strong, weak, or probabilistic sense) are available to optimally solve the motion-planning problem in 2- and 3-D and even in higher or non-Euclidean dimensional spaces [1]–[4], with respect to various requirements such as path length or kinodynamic constraints (see [5], [6] and the references therein). However, the second issue is equally important. The following three main problems must be addressed to allow robots to deal with realistic environments: 1) Assuming that the workspace is accurately known and predictable *in advance* is unrealistic. On the contrary, it will likely change from the trajectory planning phase to actual navigation, thus requiring the incorporation of *a priori*, as well as *dynamic*, knowledge about the environment [7], [8]; 2) measurements are not perfect. Real sensors and actuators are characterized by intrinsic errors which must be dealt with; and 3) decisions must be taken according to the current situation. If the adopted algorithms require the manipulation of huge internal representations (e.g., Configuration Space or similar [5]), the resulting computational load is often incompatible with the real-time requirements of highly dynamic environments.

Historically, to cope with these issues, different frameworks have been proposed in the literature. A possible classification, which takes the nature of the exploited representation into account, divides them in two classes, namely, *global* and *local* approaches. The first class, nowadays, refers to the framework of statistical estimation to provide the robot with "the most accurate" representation of the workspace "as possible" in the presence of incomplete data and sensor noise (for a comprehensive survey, see [9]). The second class adheres to a *locality principle*, thus adopting reactive, sensor-driven, and, often, suboptimal behaviors, which may or may not take inspiration from navigation capabilities of biological beings [10], [11], thus possibly giving up completeness to gain efficiency, real-time responsiveness, and a superior robustness to noise [12]–[19]. Local approaches are motivated by the fact that *huge internal representations* are difficult to maintain in terms of computational cost, reliability, and modeling issues [20]. Specifically, discussions about the use of minimal *information spaces* for sensing in mobile robots have been recently carried out in [21] and [22]. As a matter of fact, having minimal sensing requirements can play a fundamental role in meeting technological constraints, from small robotic cleaners to rovers for planetary exploration. In this sense, it seems reasonable to investigate *what* can be achieved by a mobile robot provided with minimal sensing capabilities.

This paper presents μNav, a novel navigation framework able to find a path to the goal in very complex environments without using any spatially grounded representation or self-localization capabilities. In a sense, it belongs to the group of purely reactive approaches, since the sensory information it requires comprises only of the following: 1) the azimuth direction to the goal and 2) the distance to the closest obstacle (as in classical artificial potential field (APF) algorithms [23]). However, by operating on a limited number of internal quantities, which are based solely on actual sensor data, it manages to adapt its overall behavior to the peculiarity of the environment and to find a way to the goal in each situation, given the necessary amount of time.

Section II introduces the relevant literature needed to ground benefits and drawbacks of μNav with respect to state-of-the-art navigation frameworks. Sections III–VI describe μNav in detail, providing theoretical insights. Section VII describes experimental results performed both in simulation and with two real robots, a three-wheeled mobile platform, and the hexapod robot Sistino. Conclusions then follow.

Manuscript received September 7, 2007; revised April 3, 2008 and July 9, 2008. First published December 9, 2008; current version published January 15, 2009. This paper was recommended by Associate Editor S. E. Shimony.

The authors are with the Department of Communication, Computer, and System Sciences, University of Genova, 16145 Genova, Italy (e-mail: fulvio.mastrogiovanni@unige.it; antonio.sgorbissa@unige.it; renato.zaccaria@unige.it).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TSMCB.2008.2004505

II. Related Work

A. Search for Minimal Information

Historically, a tradeoff has been sought between the amount of information needed to solve a given task and its processing burden over the related computational and representation requirements. The use of minimal amounts of information has been explored in such seminal works as [24] and [25], which paved the way for subsequent research in efficient computational systems. In robotics, there are three works that are the widely recognized milestones in this direction. The first is the work described in [26], which provides a comprehensive view about theory and applications of sensors for mobile robots. The second can be found in [27], which investigates the active role of information in robotic systems. This work introduces a theoretical classification of sensory information, thereby defining the notion of "reduction"—sensor data of a given type can be reduced to other simpler types based on selected measurement scales. Finally, the last one is provided by [28], which introduces a theoretical approach to sensor design and evaluation, based on a mapping between an abstract "sensor space" and actions representation.

With respect to robot navigation purposes, further achievements have been accomplished only in later years. An interesting classification scheme has been discussed in [20], which proposes two evaluation criteria, namely, *path length* and *safety*. Indeed, these two criteria have been the basis of a two-decade research agenda which led to ever improving competitive strategies for target reaching (see [29]–[33] and the references therein). However, given our interest in minimal information usage, the work described in [21], [34]–[36] deserves special mention. A taxonomy aimed at classifying mobile robotics systems "as a whole" is introduced in [21] and [35]. The key idea is to find *universal rules* amenable for application to many robotics-related tasks, such as localization, navigation, and planning. Although several parameters have been selected, what is actually compared is the resulting *information space*, i.e., the information content that is effectively provided by sensors. An interesting outcome of this research is that some systems are "more informed" than others when carrying out their tasks. As a consequence, a "dominance" relationship is introduced, which is very similar to "reduction" as defined in [27]; robotic systems can be compared based on their information space, and in particular, specific subsystems can be evaluated, e.g., navigation in case of mobile robots. To date, this approach represents the most accurate and principled framework to compare navigation algorithms with respect to their minimum number of *information units* which are necessary to solve the navigation problem.

Other contributions address the issue of *what* can be achieved given a limited information space. In the following paragraphs, we briefly introduce and discuss a limited number of algorithms which take the role of minimal information for navigation into account.

The seminal algorithms *Bug1/2* [37], [38] formalize the common-sense idea that a goal can be reached by alternating two simple behaviors, i.e., "move toward the goal" as long as you can and, when an obstacle is encountered, "follow the obstacle boundary" until it is possible to "move toward the goal" again. Path planning can thus be regarded as a continuous online process which combines locality with a globally convergence criterion, i.e., to head to the goal whenever possible. Similar minimalist principles led also to the design of the *Class1* algorithm [39], [40]. Buglike algorithms have been used to support multirobot exploration strategies; *PBug1* and *MRBug* [41] exhibit an optimal competitiveness with respect to the time required to reach the goal and the usage of constant memory to store relevant information. In all these cases, robots are assumed to have an ideal self-localization capability, to be able to measure the Cartesian distance between any two points and to be endowed with a contact sensor to detect obstacles, which are assumed to be stationary. Under these assumptions, these algorithms are guaranteed to be complete, i.e., a path to the goal is always found, subject to the fact that such a path exists. Furthermore, upper bounds can be easily determined on the basis of pure geometrical considerations, such as the distance between start and goal locations and the lengths of obstacle boundaries.

If the contact sensor is replaced by an infinite range sensor with a 360° infinite orientation resolution, then *TangentBug* [42], [43] is able to find the shortest paths to the goal than those computed by Bug1/2. Specifically, TangentBug exploits a *local tangent graph* in order to choose the local optimal direction with respect to the target. However, the robot is assumed to be able to detect discontinuities in range data (otherwise called *gaps* in the recent literature [22], [34]) using an *ideal* sensor in the sense of [28], and—above all—the maximum sensor range is assumed to be potentially infinite in length, which is clearly unrealistic. The first requirement can be partially dealt with by adopting state-of-the-art techniques for data association, whereas the second one is usually managed by adopting a finite-resolution range sensor, such as a laser rangefinder, therefore possibly yielding to suboptimal paths. A similar approach is considered in *VisBug21/22* [44], [45], which investigates the role of finite range sensing with respect to path optimality. It is demonstrated that VisBug21/22 always generate paths which are not worse than those obtained with Bug2.

DistBug [20] is characterized by a better exploitation of range data. Specifically, the algorithm assumes that a finite range sensor is used to define an improved leaving condition from the "follow the obstacle boundary" phase. Although this approach leads to paths which are shorter, in average, with respect to those provided by Bug1/2, perfect knowledge of the robot position in the workspace is assumed when following boundaries. An extension to classical Buglike algorithms is introduced in [46]; *SensBug* is able to deal with dynamic environments, under the strong assumption of being able to understand whether sensed obstacles are stationary or moving.

In spite of these strong limitations, variants of Bug algorithms are still presented in the research literature with success [8]. For instance, they have been used in pursuit-evasion scenarios to search for moving targets, as described in [47] and [48]. In [49], a Buglike approach is adopted, which integrates a dynamic APF framework with space decomposition. The algorithm is able to define subgoals to be reached in sequence; however,

it cannot formally guarantee their reachability. Laubach and Burdick [50], [51] propose *RoverBug* and *WedgeBug*, two algorithms specifically thought for planetary exploration. Recently, Magid and Rivlin [52] introduced a variant of TangentBug called *CautiousBug*, which outperforms the original algorithm by executing a *cautious* search when deciding which direction to follow on obstacle boundaries. A new variant called *LadyBug* has been recently presented in [53], which incorporates bioinspired heuristics to improve the robot trajectory in real time.

Buglike frameworks do not exhaust the minimalist navigation literature. Other algorithms explore the role of "angular" information in mazelike workspaces. Blum and Kozen [54] reported one of the very first attempts in describing the role of the compass sensor for a finite automaton escaping from 2-D mazelike environments. The first minimalist and sound strategy to escape from mazelike environments has been presented in [55] and [56] and then refined in [57]; the *Pledge* algorithm is able to leave a polygonal maze, measuring only its turning angles. However, two issues must be considered. The first is that perfect measurements are assumed in the original formulation, although Kamphans and Langetepe [57] argue that if measurement errors are "small enough," Pledge still behaves correctly; the second is that Pledge is able solely to *escape from mazes*—it is not able to reach a target position at all, located inside or outside the maze itself. The first algorithm to incorporate directional information for nonheuristic maze search and navigation is *Angulus* [58]. Unfortunately, in this case, an unrealistic assumption is posed as well. Angulus must be informed of the azimuth direction of the robot with respect to a frame located in the goal.

A number of algorithms specifically address the tradeoff between minimal information and improved efficiency. *Alg1/2* [59], [60] are aimed at improving Bug2 behavior by storing, in a stacklike structure, the sequence of *hit points* occurring within an actual path to the goal, thus being able to choose the opposite direction to follow an obstacle boundary when a *hit point* is encountered for the second time. Similarly, *HD-I* and *Ave* [61], [62] are able to learn whether to follow, clockwise or counterclockwise, an obstacle boundary by trial and error.

Finally, uncertainty in motion and sensing in the context of limited information is first addressed in [63] and, in recent years, by the work described in [34] and [57], among others.

B. Possible Classification of Navigation Frameworks

In this section, a simple classification of navigation frameworks is attempted. Specifically, we adhere to the nomenclature proposed in [21], [35], and [36] to formally compare robotic systems using *information spaces*. However, the proposed investigation considers solely the number of *information units* which are strictly necessary for actual algorithms to operate; put differently, we do not take the *path length* or other quality measures into account. Therefore, we refer to such classes as $\mathcal{I}_n$, where n denotes the number of required information units given an algorithm a_n, $a_n \in \mathcal{I}_n$ if it is able to reach a target using n *information units*. In the following paragraphs, we limit our investigation to lower n values.

Algorithms belonging to $\mathcal{I}_6$ include TangentBug, VisBug21/22, and HD-I. TangentBug is able to detect *discontinuities* in range data, thus being able to select which one is better to chase in order to reach the target. It necessitates the measurement of distance vectors both to the left and to the right discontinuity points (each vector being 2-D), as well as its own Cartesian position, in order to compute the distances between discontinuity points and the target, which sums up to six information units. Similar considerations hold for RoverBug, WedgeBug, and CautiousBug, which reason differently about the discontinuities, as well as for LadyBug. VisBug21/22 adopts a similar approach, in that it is assumed to recognize *gaps* in the field of view, and it uses this information to outperform Bug2. Finally, HD-I and Ave exploit similar considerations related to distances to learn better policies for navigation.

SensBug belongs to $\mathcal{I}_{4+}$. In principle, it assumes to know the distance and the direction of the target, the distance vector to the closest obstacle, and the closest obstacle *type* (i.e., static or moving obstacle), thus requiring five information units. However, since the obstacle type is assumed to be a simple Boolean one, it weighs less with respect to other *ordered* information units.

Bug1/2, Class1, Alg1/2, PBug1, and Angulus belong to $\mathcal{I}_3$. Specifically, Bug1/2 (and their many variants, including Class1 and Alg1/2) require to know at least the unit normal vector to the obstacle boundary measured through an "ideal" touch sensor (i.e., one information unit), as well as the robot position within the workspace (i.e., two information units), to compute the positions of *hit points* and their distances with respect to the target. However, a realistic implementation of Bug1/2 belongs to $\mathcal{I}_4$, since the "ideal" touch sensor must be implemented through some sort of range sensor, which returns the distance to the obstacle's boundary in addition to the unit normal vector. If this information is available, DistBug can perform better by computing the distance between the closest obstacle and the target, whereas in Angulus, a completely different approach is considered, i.e., the use of a couple of angular measurements (i.e., one information unit each) plus an "ideal" touch sensor (or a more realistic range sensor) for obstacle avoidance.

As shown in the following, μNav belongs to $\mathcal{I}_3$ when relying on a range sensor for obstacle avoidance, whereas it belongs to an even simpler class $\mathcal{I}_2$ when assuming an "ideal" touch sensor. In the authors' knowledge, μNav and Pledge are the only algorithms belonging to $\mathcal{I}_2$, since they exploit—respectively—the knowledge about the direction of the target and compass measurements. However, Pledge is intended to escape from mazes and not to reach a goal location.

III. μNav Basic Motion Law

μNav deals with the general problem of a mobile robot which must navigate between two locations $\mathbf{x_S}$ and $\mathbf{x_T}$ in a workspace $\mathbb{W} \subset \Re^2$ where obstacles are present. It is assumed that the Cartesian position of the robot with respect to $\mathbf{x_S}$ or $\mathbf{x_T}$ is never known; instead, the only information returned by sensors (i.e., the direction to the goal and the distance to the closest obstacle) is expressed with respect to a moving frame F_R centered on the

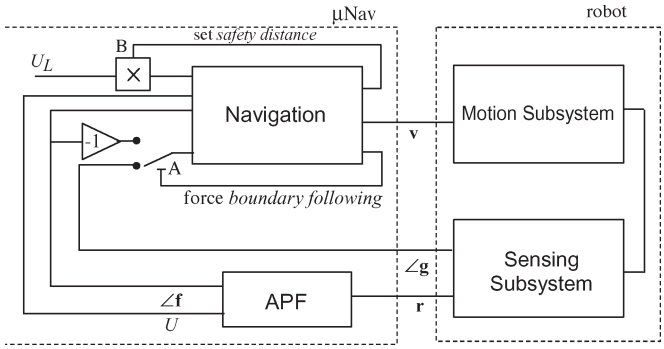

Fig. 1. Block diagram of the system.

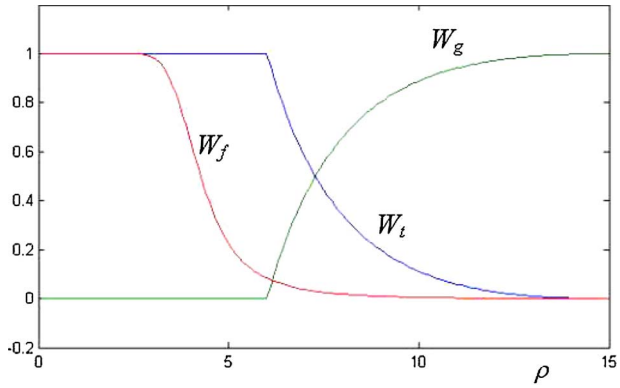

Fig. 2. Plot of W_g, W_t, and W_f versus the sensed range ρ.

robot. To achieve this, μNav computes a navigation function on the basis of an APF [23]. More formally, it is defined as follows.

Definition 1: The following time-varying quantities are defined with respect to F_R (for simplicity, their dependence on time is written explicitly only when they are first introduced).

1) $\mathbf{g}(t) = (g_x, g_y)$ is a unit vector directed to the goal.
2) $\mathbf{r}(t)$ is the distance vector to the closest obstacle, where $\angle\mathbf{r}$ corresponds to the direction of the range measurement and $\rho = |\mathbf{r}|$ corresponds to the sensed range.
3) $U(t)$ is the current value of the potential field, i.e.,

$$U = \begin{cases} \left(\frac{1}{\rho} - \frac{1}{\rho_{\max}}\right)^2, & \rho \le \rho_{\max} \\ 0, & \rho > \rho_{\max} \end{cases} \tag{1}$$

where $\rho_{\max}$ identifies the maximum sensing range.
4) $\mathbf{f}(t) = (f_x, f_y)$ is a unit vector directed along the inverse gradient of the potential field; since $-\nabla U$ is directed along $\angle\mathbf{r}$ but with an opposite sign, $\mathbf{f}$ can be computed as

$$\mathbf{f} = \begin{cases} -\frac{\nabla U}{|\nabla U|}, & \nabla U \neq 0 \\ 0, & \nabla U = 0 \end{cases}$$

$$\nabla U = \frac{d}{d\rho} U \cdot \left(\frac{\mathbf{r}}{|\mathbf{r}|}\right)$$

$$\frac{d}{d\rho} U = \begin{cases} \frac{2}{\rho^2 \rho_{\max}} - \frac{2}{\rho^3}, & \rho \le \rho_{\max} \\ 0, & \rho > \rho_{\max}. \end{cases} \tag{2}$$

5) $\mathbf{t}_1(t)$ and $\mathbf{t}_2(t)$ are unit vectors referred to as the "left" and "right" tangents, respectively. Both of them are tangents to the APF equipotential line, i.e.,

$$\begin{aligned} \mathbf{t}_1(t) &= (t_{1x}, t_{1y}) = (f_y, -f_x) \\ \mathbf{t}_2(t) &= (t_{2x}, t_{2y}) = (-f_y, f_x). \end{aligned} \tag{3}$$

As is usual in many APF-based approaches, 1) and 2) in Definition 1 are directly returned by sensors at time t (see the block diagram of the system in Fig. 1), whereas 3), 4), and 5) are locally computed on the sole basis of 1) and 2), without needing a map of obstacles or any other global representation. Instead of the distance vector $\mathbf{r}$ (which is 2-D), one can assume an "ideal" touch sensor (in the sense of [28]) which returns only the unit normal vector to the obstacle's boundary (which is a 1-D quantity). The following discussion can be easily adapted to this situation, by assuming that the robot "touches" the obstacles instead of keeping a safety distance, without invalidating the general principle.

In contrast with standard APF, instead of descending the gradient the robot motion law, $\mathbf{v}(\mathbf{t})$ is given by

$$\mathbf{v} = [W_g(U)\mathbf{g} + W_t(U)\mathbf{t} + W_f(U)\mathbf{f}]\, V_{\text{ref}} \tag{4}$$

where V_{ref} is a reference value for the speed and $\mathbf{t}$ is one of the two tangent vectors, namely, $\mathbf{t}_1$ or $\mathbf{t}_2$. The general properties of μNav do not change depending on the strategy for choosing $\mathbf{t}$, e.g., either it is chosen *a priori*, randomly during navigation, or alternating between $\mathbf{t}_1$ and $\mathbf{t}_2$. In the current implementation, μNav chooses the tangent vector which is closer to the current direction of motion when approaching the obstacle (i.e., which x component in the moving frame F_R is maximum), which allows one to avoid rough turns

$$\mathbf{t} = \mathbf{t}_i, \qquad i = \underset{m=1,2}{\arg\max}(t_{mx}). \tag{5}$$

The speed vector $\mathbf{v}$ results from the composition of $\mathbf{g}$, $\mathbf{t}$ and $\mathbf{f}$, weighted by the three weights W_g, W_t, and W_f, which—on their turn—depend on U, i.e., the current value of the APF. The basic motion law in (4) implements a simple but effective navigation strategy, as shown in the following: 1) when the robot is far from obstacles, it heads to the goal, and 2) when the robot is approaching an obstacle, it follows its boundary (on an equipotential line of the APF) until it can safely head to the goal again. Specifically, by defining U_L as the equipotential line to navigate on to avoid obstacles, W_g and W_t are set as follows:

$$W_g(U) = \begin{cases} 1 - \frac{U}{U_L}, & U \le U_L \\ 0, & U > U_L \end{cases} \tag{6}$$

$$W_t(U) = \begin{cases} \frac{U}{U_L}, & U \le U_L \\ 1, & U > U_L. \end{cases} \tag{7}$$

The choice of W_f is not as critical as W_g and W_t. For example, it can have a sigmoidal profile such as in Fig. 2

$$W_f(U) = \frac{1}{1 + (\lambda_1 U)^{-1} e^{(U_L - \lambda_1 U)\lambda_2}} \tag{8}$$

where λ_1 and λ_2 determine the exact shape of the sigmoid. Other choices are equally valid for W_f. The only constraint is

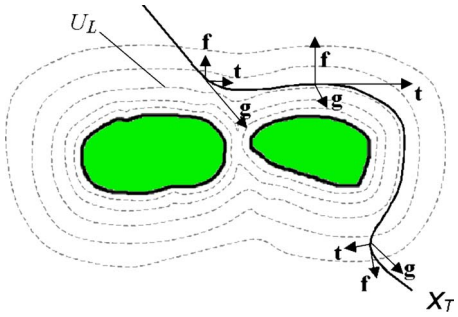

Fig. 3. Robot trajectory in the APF.

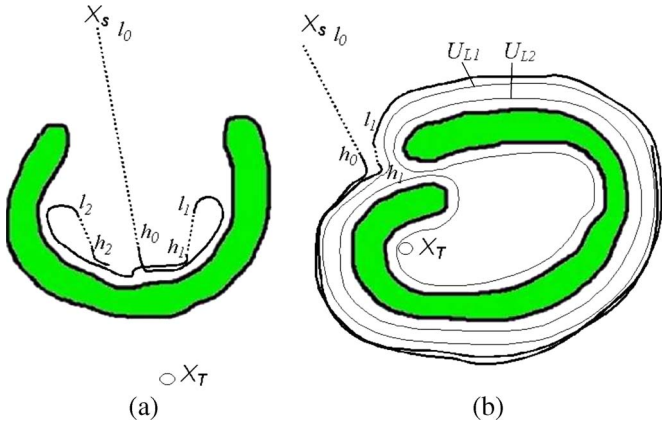

Fig. 4. (a) Robot trapped in a concavity. (b) Robot blocked by obstacles which surround the goal.

that the two weights W_g and W_t must be the most important ones in determining the motion law, whereas W_f must become relevant only when the robot is very close to obstacles. Qualitatively, the overall behavior is as follows (Fig. 3): 1) When the robot is far from obstacles, since $U \approx 0$ holds, *heading to the goal* is predominant because $W_g \approx 1$ and $W_t \approx 0$; 2) when approaching an obstacle, since $U > 0$ holds, both *boundaries* following and *heading to the goal* contribute to the robot's motion because $W_g > 0$ and $W_t > 0$; and 3) as the robot gets closer to the obstacle, U tends to U_L until its trajectory lies completely on the equipotential line; the *boundary following* is predominant because $W_g \approx 0$ and $W_t \approx 1$. When navigating on the equipotential line, the sigmoidal shape of W_f prevents the robot from *climbing* the APF to higher values; this event could happen either because of errors in motion control or as a consequence of moving obstacles. In addition, W_f helps the robot leave the equipotential line whenever it is possible to safely head to the goal.

In contrast with other APF-based approaches, the law of motion described in (4) prevents the robot from being trapped into local minima because equipotential lines are closed lines with neither drains nor sources. Unfortunately, when obstacles have more complex shapes, the same law of motion does not prevent the robot from being trapped into loops. Two cases of simple loops are shown in Fig. 4. In analogy with the terminology introduced by Buglike algorithms, h_i is referred to as the ith "hit point"; when approaching an obstacle, it corresponds to the first point in the free space where U becomes greater than zero, thus enabling boundary following. Similarly, l_i is referred to as the ith "leave point"; when leaving an obstacle, it corresponds to the first point where U decreases to zero, thus letting the robot freely head to the goal. It is important to outline that we refer to hit and leave points only to explain the behavior of the system but that they are *never* stored as discrete decision points to search a path to the goal.

In Fig. 4(a), the robot heads toward the goal in $l_0 \rightarrow h_0$, it follows an equipotential line in $h_0 \rightarrow l_1$, it heads to the goal in $l_1 \rightarrow h_1$, it follows the boundary in $h_1 \rightarrow l_2$, and so on until it ends up in l_1 again. Its trajectory will lie on a closed curve "forever." In order to escape from the concavity, the robot should recognize the loop and decide to follow the boundary of the obstacle instead of heading to the goal in l_1 or l_2. In Fig. 4(b), the robot is blocked by obstacles which surround the goal; the robot heads to the goal in $l_0 \rightarrow h_0$, it follows the boundary in $h_0 \rightarrow l_1$, it heads to the goal in $l_1 \rightarrow h_1$, and so on until it ends up in l_1. Again, the robot is trapped in a loop. Notice that, in this case, the problem is twofold. First—as in the previous case—the robot should decide to follow the boundary of the obstacle in the proximity of l_1. Second, it should raise the value of U_L from U_{L1} to U_{L2}. In fact, the presence of saddles in the equipotential field can prevent the robot from finding a path to the goal through narrow passages, since U_L is initially assigned a low value as it determines the safety distance from obstacles.

In general, a trajectory $\mathbb{T}$ generated by μNav's law of motion can be always decomposed into an alternating sequence of the following: 1) *free segments*, i.e., straight lines oriented toward the goal, and 2) *obstacle segments*, i.e., equipotential curves such that $U \approx U_L$, where the transitions between them are smoothed by the values of W_g, W_t, and W_f. More formally, these are defined as follows.

Definition 2: A *free segment* F_i is a rectilinear piece of trajectory in the free space (i.e., $U = 0$ for all points in F_i) heading to the goal, starting in a leave point l_i and ending in a hit point h_i.

Definition 3: An *obstacle segment* O_i is a piece of trajectory which follows the boundary of obstacles along an equipotential line (i.e., $U > 0$ for all points in O_i), starting in a hit point h_i and ending in a leave point l_{i+1}. In special cases, O_i can have an infinite length (i.e., it does not end up in a leave point).

Definition 4: A μNav trajectory $\mathbb{T}$ is an alternating sequence of *free* and *obstacle* segments. $\mathbb{T}$ always starts with a free segment F_0, where l_0 coincides with the robot starting point; if a path to the goal is found, $\mathbb{T}$ contains a free segment F_n whose endpoint h_n coincides with the goal. In special cases, $\mathbb{T}$ can include, as a last segment, an *obstacle segment* of infinite length.

Definition 5: A μNav loop $\mathbb{T}_{\text{loop}}$ is a closed μNav trajectory, i.e., starting with a *free segment* F_i and ending with an *obstacle segment* O_{i+k} such that $l_{i+1+k} = l_i$. We say that the robot is trapped into $\mathbb{T}_{\text{loop}}$ when its trajectory $\mathbb{T}$ tends to almost lie on $\mathbb{T}_{\text{loop}}$ as $t \rightarrow \infty$ (the adverb "almost" informally takes control errors into account, as well as slight variations in U_L in subsequent loops).

In the following sections, the navigation strategies adopted by μNav to avoid loops of whatever complexity are described. Fig. 1 shows that this is done by operating in the following two

ways: 1) by modifying the basic motion law in order to force boundary following when required through switch A, and 2) by modifying U_L in order to explore narrow passages through gain B. To this purpose, the system must be able to recognize that some conditions are satisfied; these conditions will be discussed in the next section.

It is worth noting that, when switching to boundary following, the motion law becomes

$$\mathbf{v} = [-W_g(U)\mathbf{f} + W_t(U)\mathbf{t} + W_f(U)\mathbf{f}]\, V_{\text{ref}} \tag{9}$$

i.e., it eliminates the joint effect of $\mathbf{g}$ and $\mathbf{f}$ when $\mathbf{f} \cdot \mathbf{g} > 0$ by substituting $W_g\mathbf{g}$ with a component $-W_g\mathbf{f}$ which counterbalances $W_f\mathbf{f}$ (Fig. 1).

IV. Basic Deadlock Detection

In principle, loops could be avoided by recognizing previously visited hit or leave points. Unfortunately, the recognition process would require an accurate self-localization system, which is in contrast with our assumptions. To avoid this, μNav relies on the idea of defining an indicator whose value necessarily increases whenever the robot is trapped in a loop.

Definition 6: The following quantities are introduced by assuming that positive angles increase counterclockwise.

1) $\gamma(t) = \angle\mathbf{g}(t)$ is the direction of the goal with respect to F_R.
2) $\phi(t) = \angle\mathbf{f}(t)$ is the direction of the repulsive force with respect to F_R.
3) The "loop detector" $\hat{\alpha}(t)$ is defined as

$$\hat{\alpha}(t) = \int_0^t -\dot{\gamma}(\tau) \cdot s\left(\phi(\tau)\right) d\tau \tag{10}$$

which requires $\dot{\gamma}(t)$ to be at least piecewise continuous, a condition which is always satisfied since the imposed μNav trajectory $\mathbb{T}$ is C^0 (and the actual robot trajectory is C^1 for obvious physical reasons). $s(\phi(t))$ is an expression of the algebraic sign of $\phi(t)$

$$s\left(\phi(t)\right) = \begin{cases} \frac{\sin(\phi(t))}{|\sin(\phi(t))|}, & \phi(t) \neq 0 \\ 1, & \phi(t) = 0. \end{cases} \tag{11}$$

The quantity $\hat{\alpha}$ in (10) represents the direction of $\mathbf{g}$ (possibly with an inverted sign); however, its values can range, in contrast with γ, between $(-\infty, \infty)$, i.e., $\hat{\alpha} = \gamma \pm 2k\pi$, with k integer positive. $\hat{\alpha}$ is a tentative indicator; a new indicator α will be soon introduced, which shares the general behavior of $\hat{\alpha}$ but is characterized by additional properties. For the moment, $\hat{\alpha}$ allows a simpler explanation. In (11), $s(\phi)$ depends exclusively on the tangent vector $\mathbf{t}$ chosen to avoid obstacles. If the left tangent $\mathbf{t}_1$ is chosen, the robot turns on the left when approaching obstacles, therefore keeping obstacles to its right side until it heads to the goal again ($s(\phi)$ is positive); if the right tangent $\mathbf{t}_2$ is chosen, the opposite is true. As shown in the following, including $s(\phi)$ in (10) allows one to obtain an identical behavior for $\hat{\alpha}$, whichever tangent is chosen.

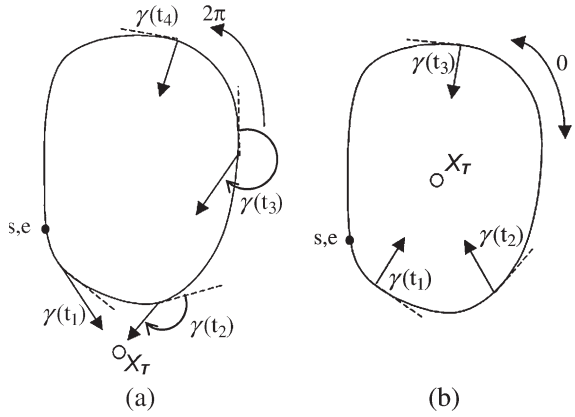

Fig. 5. Behavior of $\hat{\alpha}$ along a closed curve which (a) does not contain and which (b) contains the goal.

The behavior of $\hat{\alpha}$ along a μNav trajectory $\mathbb{T}$ is formally shown through the following theorems.

Theorem 1: If l_i and h_i are the endpoints of a *free segment* F_i, then $\hat{\alpha}_{h_i} - \hat{\alpha}_{l_i} = 0$.

Proof: By definition, since a *free segment* is a straight line heading to the goal. ■

Theorem 2: If O_i is an *obstacle segment* with endpoints h_i and l_{i+1} preceded by F_i and followed by F_{i+1}, then $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i} = \pm 2k\pi$, with k integer positive.

Proof: By definition, since $\gamma_{h_i} = \gamma_{l_{i+1}} = 0$. In fact, h_i and l_{i+1} belong, respectively, to the *free segments* F_i and F_{i+1}, and $\hat{\alpha}_{l_{i+1}} = \gamma_{l_{i+1}} \pm 2k\pi$, $\hat{\alpha}_{h_i} = \gamma_{h_i} \pm 2k\pi$. ■

Theorem 3: Given the following: 1) closed C^0 trajectory $\mathbb{T}$ with a single loop which does not contain the goal [Fig. 5(a)]; 2) $s(\phi)$ is constant along $\mathbb{T}$; and 3) two arbitrary points s and e on the curve such that $s = e$, if the robot moves along the $\mathbb{T}$ counterclockwise, then $\hat{\alpha}_e - \hat{\alpha}_s = 2\pi \cdot s(\phi)$; if the robot moves along $\mathbb{T}$ clockwise, then $\hat{\alpha}_e - \hat{\alpha}_s = -2\pi \cdot s(\phi)$.

Proof: It is immediate from simple geometric considerations, e.g., notice in Fig. 5(a) that when looping counterclockwise, $\gamma(t)$ decreases and hence, $\hat{\alpha}(t)$ increases when $s(\phi) = 1$ and decreases otherwise. ■

Theorem 4: Given the following: 1) simple C^0 closed trajectory $\mathbb{T}$ with a single loop which contains the goal [Fig. 5(b)]; 2) $s(\phi)$ is constant along $\mathbb{T}$; and 3) two arbitrary points s and e on the curve such that $s = e$, then $\hat{\alpha}_e - \hat{\alpha}_s = 0$ when the robot moves along $\mathbb{T}$, either clockwise or counterclockwise.

Proof: It is immediate from simple geometric considerations. ■

Theorem 5: Given the following: 1) simple C^0 closed trajectory $\mathbb{T}$ with multiple loops (Fig. 6); 2) two arbitrary points s and e on the curve such that $s = e$; and 3) each loop is followed by the robot either entirely clockwise or counterclockwise and $s(\phi)$ is constant along the curve, then $\hat{\alpha}_e - \hat{\alpha}_s$ can be computed as the sum of the following contributions: 1) for each loop which does not contain the goal Theorem 3 holds, and 2) for each loop which contains the goal Theorem 4 holds.

Proof: From simple geometric considerations. By setting $s = s_1$ and $e = e_2$, the case in Fig. 6(a) is immediate, since the trajectory $s_1 \rightarrow e_2$ can be decomposed into two loops $s_1 \rightarrow e_1$ and $s_2 \rightarrow e_2$ which have the same tangent in s_1, e_1, s_2, and e_2 and for which Theorems 3 and 4 can be applied separately.

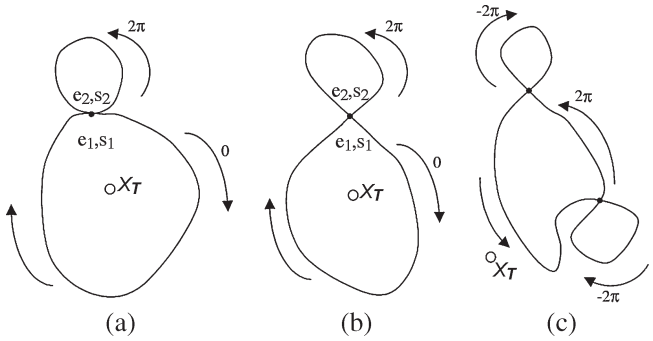

Fig. 6. Behavior of $\hat{\alpha}$ along a closed curve with multiple loops.

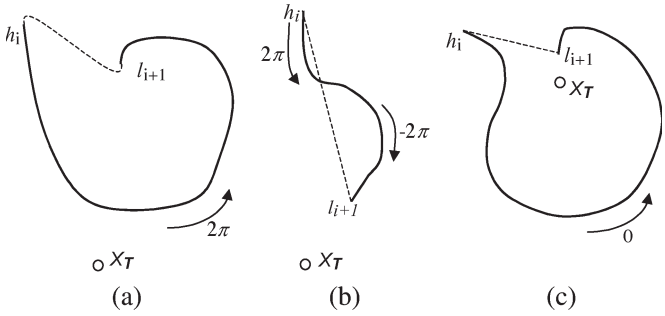

Fig. 7. Behavior of $\hat{\alpha}$ along an *obstacle segment*.

The cases shown in Fig. 6(b) and (c) are less immediate but can be easily verified by considering that $\gamma_{s_2} = \gamma_{e_1}$ and $\gamma_{s_1} = \gamma_{e_2}$ hold and, hence, when considering $s_1 \rightarrow e_2$

$$\begin{aligned}\alpha_{e_2} - \alpha_{s_1} &= (\alpha_{e_1} - \alpha_{s_1}) + (\alpha_{e_2} - \alpha_{s_2}) \\ &= [0 - (\gamma_{s_1} - \gamma_{e_1})] + [2\pi - (\gamma_{s_2} - \gamma_{e_2})] \\ &= 2\pi \end{aligned} \tag{12}$$

i.e., $\alpha_{e_2} - \alpha_{s_1} = 2\pi$ can be computed by assuming that the robot has traveled along each loop separately and by subtracting the quantities $(\gamma_{s_1} - \gamma_{e_1})$ and $(\gamma_{s_2} - \gamma_{e_2})$, corresponding to the pieces $e_1 \rightarrow s_1$ and $e_2 \rightarrow s_2$, which are not really part of the trajectory. ■

Theorem 6: Given an *obstacle segment* O_i with endpoints h_i and l_{i+1}, Theorem 5 can be adopted to compute $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i}$ by connecting l_{i+1} to h_i with an arbitrary curve, such that $\hat{\alpha}_{h_i} - \hat{\alpha}_{l_{i+1}} = 0$, and by applying Theorem 5 to the resulting closed curve.

Proof: Since an *obstacle segment* is such that $\gamma_{h_i} = \gamma_{l_{i+1}} = 0$ for Theorem 2, it is always possible to connect l_{i+1} to h_i with an "S-shaped" curve (which can be drawn as a straight line for simplicity) in order to obtain a C^0 closed curve. See Fig. 7 (obstacles are not drawn for clarity of representation). Along the "S-shaped" curve, the overall variation of $\hat{\alpha}$ is null. Since $s(\phi)$ is constant along O_i, Theorem 5 can be applied to the closed curve, and the total variation of $\hat{\alpha}$ is exclusively due to O_i. ■

Theorems 3–6 are very important in that they allow one to foresee the behavior of $\hat{\alpha}$ in all cases. Consider, once again, Fig. 4; $\hat{\alpha}$ is constant in $l_1 \rightarrow h_1$, it increases in $h_1 \rightarrow l_2$ (from Theorem 3, the robot is turning clockwise and $s(\phi) = -1$),

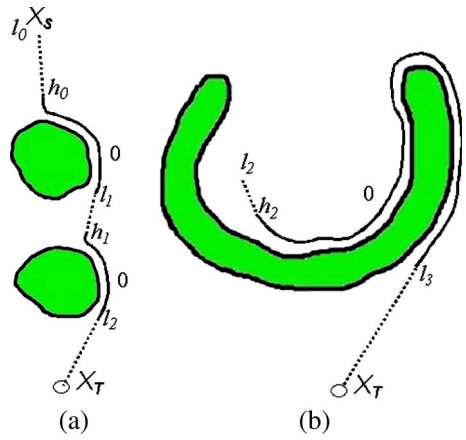

Fig. 8. (a) μNav avoiding simple convex obstacles. (b) μNav exiting from a concavity.

it is constant in $l_2 \rightarrow h_2$, and it increases again in $h_2 \rightarrow l_1$ (from Theorem 3, the robot is turning counterclockwise and $s(\phi) = 1$). The robot is trapped in a loop $\mathbb{T}_{\text{loop}}$ which does not contain the goal, and $\hat{\alpha}$ increases every loop. Fig. 8(a) shows the behavior of $\hat{\alpha}$ when avoiding simple convex obstacles; by adopting Theorem 6, it is immediately noticed that $\hat{\alpha}_{h_0} = \hat{\alpha}_{l_1} = \hat{\alpha}_{h_1} = \hat{\alpha}_{l_2}$. Whichever tangent is chosen when approaching obstacles, $\hat{\alpha}$ always initially increases (and next decreases), since it turns counterclockwise when the left tangent is chosen (and, hence, $s(\phi) = 1$) and clockwise when the right tangent is chosen (and, hence, $s(\phi) = -1$). Fig. 8(b) shows what happens if—at a given instant—the robot is able to exit from the loop in Fig. 4; $\hat{\alpha}$ initially increases and next decreases. More complex cases can be considered as well; however, it is easy to verify that the general trend is still the same. When adopting the motion law in (4), $\hat{\alpha}$ increases when the robot is "trapped" within a concavity which does not contain the goal, whereas it has a null mean value otherwise.

This suggests the implementation of the following rules to operate on the switch A in Fig. 1.

Rule 1) μNav switches, at time t, from the motion law in (4) to the one in (9) (which forces boundary following) **when** $\text{cond}_1 = \text{true}$, with $\text{cond}_1 : \mathbf{f} \cdot \mathbf{g} < 0$.

Rule 2) μNav switches, at time t, from the motion law in (9) to the one in (4) (which allows *heading to the goal*) **when** $(\text{cond}_2 \wedge \text{cond}_3) = \text{true}$, with $\text{cond}_2 : \mathbf{f} \cdot \mathbf{g} > 0$, and $\text{cond}_3 : \hat{\alpha} \leq \alpha^{\text{ub}}$. α^{ub} (the superscript stands for "upper bound") is defined as

$$\alpha^{\text{ub}}(t) = \int_{t_{h_i}}^{t} \left(|\dot{\gamma}(\tau)| + \epsilon\right) d\tau \tag{13}$$

where t_{h_i} corresponds to the most recent transition to boundary following and $\epsilon > 0$ is an arbitrarily small positive real number.

Rule 3) μNav sets, at time t

$$U_L = U_{L_0} \cdot f\left(\hat{\alpha}(t)\right) \tag{14}$$

where $f(\xi)$ is an increasing monotonic function such that $f(\xi) \rightarrow 1$, as $\xi \rightarrow 0$, $f(\xi) \rightarrow$

$U_{L\max}/U_{L_0}$, and $\xi \to \infty$; and where U_{L_0} and $U_{L\max}$ are, respectively, the minimum and maximum values for U_L.

Rule 1) simply forces boundary following when the robot is approaching an obstacle ($\text{cond}_1 = \text{true}$). Rule 2 switches back to *heading to the goal* when there are no obstacles between the robot and the goal in the immediate surroundings and the value of $\hat{\alpha}$ is smaller than or equal to α^{ub}, i.e., ($\text{cond}_2 \wedge \text{cond}_3$) = true. Rule 2) is sufficient to exit from concavities. Consider, again, the simple case in Fig. 4(a); α^{ub} is always positive, and it increases faster than $\hat{\alpha}$ by definition [compare (10) and (13)]. When the robot enters the concavity for the first time, $\hat{\alpha} = \alpha^{\text{ub}} = 0$; since α^{ub} increases faster, both cond_2 and cond_3 are verified in the proximity of l_1, and the robot is free to head toward the goal. However, the integral in (13) is computed starting from t_{h_i}, which means that α^{ub} resets to zero every loop, whereas $\hat{\alpha}$ always increases; at a given time, cond_3 is no longer verified, and the robot is forced to follow the boundary in the proximity of l_1 or l_2 until it eventually escapes the concavity. When this happens, $\hat{\alpha}$ starts to decrease again [Fig. 8(b)]; the robot is free to head toward the goal in l_3. Notice that local oscillations in the robot's trajectory, as well as the sensor noise, guarantees that α^{ub} increases faster than $\hat{\alpha}$ even when setting $\epsilon = 0$. Rule 3) raises U_L to allow the robot to move on equipotential lines which are closer to obstacles and to explore new areas where access was denied by saddles in the potential field.

Unfortunately, $\hat{\alpha}$ does not provide sufficient information to escape from loops such as those in Fig. 4(b). In fact, it neither increases nor decreases when the robot moves along a closed trajectory which contains the goal, and it is therefore inadequate to infer whether it is necessary to do the following: 1) switch to boundary following in l_1 and, even more critical, 2) raise the value of U_L in order to pass through the narrow passage. In fact, for lower U_L values, the corresponding equipotential line divides the workspace into two nonconnected components, one containing the robot and the other the goal. In this case, a path cannot be found by simply following the boundary.

This situation could be detected—at least in principle—by introducing a fixed reference frame F_T centered on the goal, and by defining, for example, $\chi(t)$ as the angle between the F_T x-axis and the straight line connecting the goal to the robot. This would allow for the introduction of a new quantity, i.e.,

$$\hat{\beta}(t) = \int_0^t -\dot{\chi}(\tau) \cdot s\left(\phi(\tau)\right) d\tau \tag{15}$$

which increases when the robot's trajectory $\mathbb{T}$ is a closed curve which contains the goal, and it is constant (on average) when it does not contain the goal. Unfortunately, χ cannot be directly measured by the robot, as it is the case for γ; thus, the solution is not really feasible.

In addition, the behavior of the system in more complex situations cannot be immediately foreseen. Switching between the motion laws in (4) and (9) can produce more complex trajectories, e.g., loops $\mathbb{T}_{\text{loop}}$ in which $\hat{\alpha}$ decreases [Fig. 9(a)] or even including smaller loops inside [Fig. 9(b)].

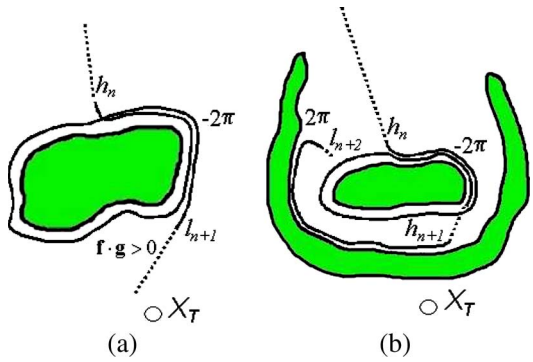

Fig. 9. (a) Loop in which $\hat{\alpha}$ decreases. (b) Loop which includes smaller loops inside.

In the next section, all these problems are faced by introducing a couple of new "loop detectors" α and β.

V. Advanced Deadlock Detection

Even if the definition of α and β can initially appear arbitrary, they should not be considered "heuristics," in that they formally guarantee—when used in Rules 2 and 3—that the robot cannot be trapped in a $\mathbb{T}_{\text{loop}}$ forever.

In particular, we want μNav to recognize the following situations which experience have revealed to be problematic for $\hat{\alpha}$ alone.

1) The robot is moving along an equipotential line U_{L1} which contains the goal [Fig. 4(b)] and from which the goal is not reachable (and, hence, U_L should be raised).
2) The robot is moving along an equipotential line which contains the goal U_{L2}, from which the goal is reachable only by switching to boundary following in the proximity of l_1 [Fig. 4(b)].
3) The robot is trapped in a bigger loop which contains smaller loops inside (Fig. 9).

The latter situation is particularly challenging since, when the robot has managed to escape from a loop, $\hat{\alpha}$ decreases to allow for simple obstacle avoidance in the remaining path to the goal; however, if it is trapped into a bigger loop which contains the previous one, the robot can find itself in a situation of equilibrium where the overall increments and decrements of $\hat{\alpha}$ sum up to zero.

Algorithm 1 `function update_alpha ()`
The function returns α depending on the current value of $\hat{\alpha}$. To this purpose, it requires to store, in memory, four state variables, namely, α_h, $\hat{\alpha}_h$, α_{acc}, and $\alpha_{\max}$, which are initialized to zero and preserve their value across subsequent function calls.

1: **if** *robot in hit point* h_i, **then**
2: $\alpha_h = \alpha$; $\hat{\alpha}_h = \hat{\alpha}$
3: **else**, **if** *robot in obstacle segment* $(h_i \to l_{i+1})$, **then**
4: $\alpha = \alpha_h + \hat{\alpha} - \hat{\alpha}_h$
5: **else**, **if** *robot in leave point* l_{i+1}, **then**
6: **if** $\hat{\alpha} - \hat{\alpha}_h = 2k\pi$ (k integer positive), **then**
7: $\Delta\alpha = \hat{\alpha} - \hat{\alpha}_h$
8: $\alpha_{\text{acc}} = \alpha_{\text{acc}} + \Delta\alpha$

9: **else**, **if** $\hat{\alpha} - \hat{\alpha}_h = 0$, **then**
10: $\Delta\alpha = 2\pi/K_1$
11: $\alpha_{\text{acc}} = \alpha_{\text{acc}} + \Delta\alpha$
12: **else**, **if** $\hat{\alpha} - \hat{\alpha}_h = -2k\pi$ (k integer positive), **then**
13: $\Delta\alpha = \hat{\alpha} - \hat{\alpha}_h$
14: **end if**
15: **if** $\alpha_{\text{acc}} \geq 2\pi$, **then**
16: $\alpha = \alpha_{\max} + \Delta\alpha + K_2$
17: $\alpha_{\text{acc}} = 0$
18: **else**
19: $\alpha = \alpha_h + \Delta\alpha$
20: **end if**
21: $\alpha_{\max} = \max(\alpha_{\max}, \alpha)$
22: **else**
23: *do nothing*
24: **end if**
25: **return** α.

Definition 6:

1) α is built through the function in Algorithm 1 such that it always increases in situations 2) and 3).
2) β is built such that it always increases in situation 1)

$$\beta(t) = \int_0^t -\sin(\gamma(\tau)) \cdot s(\phi(\tau))\, d\tau. \tag{16}$$

Lines 1 and 2 in Algorithm 1 state that when starting to follow the boundary of the generic *obstacle segment* O_i, α_h and $\hat{\alpha}_h$ are reset, respectively, to the current values of α and $\hat{\alpha}$. Lines 3 and 4 guarantee that when the robot is moving along O_i, α increases exactly as $\hat{\alpha}$. Finally, when the robot is in l_{i+1}, Lines 5–21 update α to a value which—under some circumstances—differs from $\hat{\alpha}$. In particular, if the condition in Line 6 is verified, the condition in Line 15 is verified as well; Lines 7, 16, and 21 are sufficient to guarantee that as long as $\hat{\alpha}$ keeps increasing for a set of subsequent *obstacle segments*, α increases as $\hat{\alpha}$ (K_2 is a random positive value which can be ignored for the moment). If the condition in Line 12 is verified, the condition in Line 15 is *not* verified; in the same way as with the previous case, Line 19 guarantees that as long as $\hat{\alpha}$ keeps decreasing for a set of subsequent *obstacle segments*, α decreases as $\hat{\alpha}$. It holds $\alpha \neq \hat{\alpha}$ only when the condition in Line 9 is verified, i.e., $\Delta\alpha = 0$ such as in situation 2), or when $\hat{\alpha}$ increases after having decreased, such as in situation 3). In the first case, α is arbitrarily increased of a quantity which depends on K_1 (Line 10) but which is $< 2\pi$ considering that $K_1 > 1$. In the second case, if at a given point $\alpha_{\text{acc}} \geq 2\pi$, μNav is very "alarmist," in that it assumes that the robot is trapped into a bigger loop, and α jumps up to $\alpha_{\max}$ in order to deal with the challenging situation (Line 21).

With respect to β, notice that even if it does not have the good properties of $\hat{\beta}$, it nevertheless shares a similar behavior. When the robot is moving on an equipotential line which contains the goal, it increases, considering that $\sin(\gamma)$ is always discordant with $s(\phi)$; when the robot is looping along a trajectory which does not contain the goal, β has an almost null mean value, since $\sin(\gamma)$ oscillates in $[-1, +1]$.

As one can expect, Rules 1, 2, and 3 are rewritten on the basis of α and β. In particular, the following descriptions are given.

Rule 1) Acts as **Rule 1** in Section IV.
Rule 2) Acts as **Rule 2** in Section IV; however, α substitutes $\hat{\alpha}$ in cond_3.
Rule 3) Acts as **Rule 3** in Section IV; this time, however

$$U_L = U_{L_0} \cdot \max(f(\alpha), f(\beta)). \tag{17}$$

We now demonstrate that Rules 1–3 guarantee that the robot cannot be trapped in a loop $\mathbb{T}_{\text{loop}}$ forever.

Theorem 7: Given that $U_L = U_{L1}$, if a path to the goal exists for U_{L1}, μNav does not allow for boundary following forever.

Proof: Considering that α^{ub} always increases faster than α when following the boundary and that it resets only when heading to the goal, there exists an instant t_1 such that $\forall t > t_1$, $\text{cond}_3 = \text{true}$. Excluding U_{L1} defines an equipotential line which divides the workspace into two nonconnected components, one containing $\mathbf{x_T}$ and the other $\mathbf{x_S}$; it exists in a region along the equipotential line such that $\text{cond}_2 = \text{true}$ as well. Considering that equipotential lines are closed curves, the robot periodically reaches this region when in boundary following, and hence, there necessarily exists a time $t_2 > t_1$ such that both conditions in Rule 2) are verified, thus switching to *heading to the goal*. ■

Theorem 8: For a given U_L, if $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i} = -2k\pi$ (k positive integer) when the robot leaves the *obstacle segment* O_i, it is necessary that for some region of O_i, $\text{cond}_3 = \text{false}$.

Proof: Suppose to connect l_{i+1} and h_i in order to apply Theorem 5. If the left tangent is chosen ($s(\phi) = 1$), the robot keeps the obstacle on its right, and α decreases clockwise; otherwise, if the right tangent is chosen ($s(\phi) = -1$), the robot keeps the obstacle on its left, and α decreases counterclockwise. In both cases, there necessarily exists a region between h_i and l_{i+1} such that $\text{cond}_2 = \text{true}$, i.e., when the robot is between the obstacle and $\mathbf{x_T}$ [Fig. 9(a)]. If the robot continues to follow the boundary, this means that $\text{cond}_3 = \text{false}$ in this region. ■

Theorem 9: Given $U_L = U_{L1}$, if a path to the goal exists for U_{L1}, Rules 1 and 2 guarantee that the robot cannot be trapped in the same $\mathbb{T}_{\text{loop}}$ forever.

Proof: Per absurdum. Suppose that, for U_L, the robot is trapped in $\mathbb{T}_{\text{loop}}$; by defining $\Delta\alpha_{\text{loop}}$ as the total variation of α when performing a complete loop, three cases are possible.

1) $\Delta\alpha_{\text{loop}} < 0$.
2) $\Delta\alpha_{\text{loop}} = 0$.
3) $\Delta\alpha_{\text{loop}} > 0$.

In the following, each case is separately considered.

Case 1) At least an *obstacle segment* O_i must be present such that $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i} < 0$. From Theorem 8, this requires that for some region of O_i, $\text{cond}_3 = \text{false}$. If α decreases every loop, the same obviously happens to α_{h_i}; when $\alpha_{h_i} < 0$, it is no longer possible that $\text{cond}_3 = \text{false}$, considering that $\alpha^{\text{ub}} > 0$ by definition. Therefore, case 1) can never happen.
Case 2) Two subcases must be considered.
2a) The loop includes only segments such that $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i} = 0$.

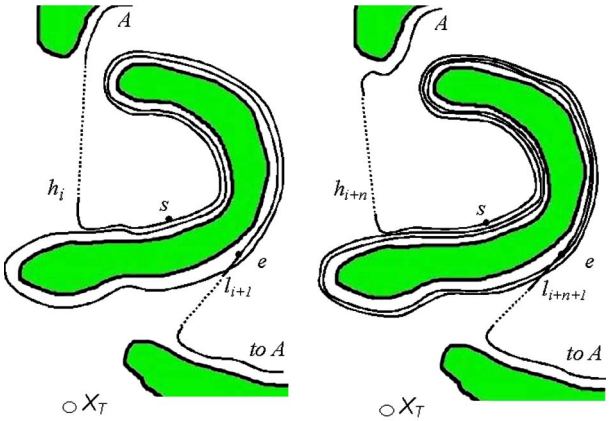

Fig. 10. μNav dealing with the same obstacle in subsequent times.

2b) The loop includes segments whose contributions sum up to zero.

Since $\mathbb{T}_{\text{loop}}$ includes at least a *free segment* from Theorem 7, in case 2a), Algorithm 1 sets $\Delta\alpha_i = 2\pi/K_1$ when $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i} = 0$ (Line 8). In case 2b), the presence of an *obstacle segment* O_i for which $\hat{\alpha}_{l_{i+1}} - \hat{\alpha}_{h_i} = -2k\pi$ (k positive integer) requires that—somewhere in the loop—a set of segments exists whose overall contribution is $2k\pi$. This means that for a given O_j, $\alpha_{\text{acc}} + \Delta\alpha_j \geq 2\pi$ and $\alpha = \alpha_{\max} + \Delta\alpha_j$ (Line 21). In cases 2a) and 2b), α increases every loop. Therefore, case 2) can never happen.

Considering that only case 3) is possible, this is sufficient to guarantee that the robot cannot be trapped forever in the same $\mathbb{T}_{\text{loop}}$. In fact, $\mathbb{T}_{\text{loop}}$ contains at least an *obstacle segment* O_i; when moving along O_i at the $(n+1)$th iteration, $\text{cond}_3 = \text{true}$ at a later time with respect to the nth iteration since α increased in the meantime, which forces the robot to follow the boundary longer and, hence, prevents it from always performing the same trajectory. ■

Theorem 9 requires that the robot is moving on an equipotential line $U_L = U_{L1}$ such that a path to the goal exists. The following theorem extends its results to the general case.

Theorem 10: If a path to the goal exists for $U_{L\max}$, Rules 1, 2, and 3 guarantee that the robot cannot be trapped in the same $\mathbb{T}_{\text{loop}}$ forever.

Proof: If the robot is looping along a closed curve, Theorem 9 guarantees that α increases every loop unless the robot is moving on an equipotential line $U_L = U_{L1}$ which divides the workspace into two nonconnected components, with one containing the robot and the other the goal. However, in this case, β increases every loop, and $U_L = U_{L0} \cdot \max(f(\alpha), f(\beta))$ increases as well. This is sufficient to state that $U_L \to U_{L\max}$ is as $t \to \infty$ and, hence, to guarantee that the robot cannot be trapped in $\mathbb{T}_{\text{loop}}$ if a path to the goal exists for $U_{L\max}$. ■

It is important to note that in a complex maze—such as environments—Theorem 10 is not sufficient to guarantee that the robot reaches the goal in a finite time, even if a path to the goal exists and the safety distance defined by $U_{L\max}$ is such that allows for finding that path. Consider Fig. 10(a). The robot starts following the obstacle in h_i and leaves in l_{i+1} after a complete loop. Unfortunately, it is not able to reach the goal and, after a long and tortuous trajectory that possibly includes subloops [not shown in the Fig. 10(a)], it ends up in the same obstacle again [Fig. 10(b)]. At this point Theorem 9 guarantees that the robot will follow the boundary of the obstacle for a longer distance; however—in unlucky conditions—this distance can correspond to two complete loops. The robot leaves in the proximity of l_{i+1} again, thus not being able to reach the goal. Strictly speaking, the robot is not looping in the same $\mathbb{T}_{\text{loop}}$, considering that the trajectory is always different; however, this undoubtedly corresponds to a deadlock situation.

To verify whether this situation is theoretically possible or not, we choose a point s along the obstacle profile, and we define α_{h_i} as the value of α in h_i; $\alpha_{1,i}$ and $\alpha_{1,i}^{\text{ub}}$ as the variation of α and α^{ub} in $h_i \to s$; α_2 and α_2^{ub} as the variation of α and α^{ub} in $s \to s$ (i.e., along the whole obstacle profile); and $\alpha_{3,i}$ and $\alpha_{3,i}^{\text{ub}}$ as the variation of α and α^{ub} between s and point e, where the condition $\alpha \leq \alpha^{\text{ub}}$ finally becomes true. When the robot is in O_i [Fig. 10(a)], the following must hold in e for cond_3 to be true:

$$\alpha_{h_i} + \alpha_{1,i} + L_1\alpha_2 + \alpha_{3,i} = \alpha_{1,i}^{\text{ub}} + L_1\alpha_2^{\text{ub}} + \alpha_{3,i}^{\text{ub}} \quad (18)$$

where L_1 is the number of complete loops around the obstacle. When the robot starts following the obstacle again in O_{i+n} [Fig. 10(b)], cond_3 becomes true when

$$\alpha_{h_{i+n}} + \alpha_{1,i+n} + (L_1+L_2)\alpha_2 + \alpha_{3,i+n} = \alpha_{1,i+n}^{\text{ub}} + (L_1+L_2)\alpha_2^{\text{ub}} + \alpha_{3,i+n}^{\text{ub}} \quad (19)$$

where $L_1 + L_2$ is the number of complete loops around the obstacle. By assuming—for simplicity—that (18) and (19) are satisfied in a point e where cond_2 is satisfied as well, the fact that the robot leaves the obstacle approximately in the same point can be written as $\alpha_{3,i+n}^{\text{ub}} \approx \alpha_{3,i}^{\text{ub}}$ and, consequently, $\alpha_{3,i+n} \approx \alpha_{3,i}$. By subtracting (18) from (19), we obtain

$$\alpha_{h_{i+n}} - \alpha_{h_i} \approx -\alpha_{1,i+n} + \alpha_{1,i} + L_2\left(\alpha_2^{\text{ub}} - \alpha_2\right) + \alpha_{1,i+n}^{\text{ub}} - \alpha_{1,i}^{\text{ub}}. \quad (20)$$

This condition, in general, can be verified for some values of the variables. The limit case (but not the only possible) is when $h_i \approx h_{i+n}$, and hence, $\alpha_{1,i+n}^{\text{ub}} \approx \alpha_{1,i}^{\text{ub}}$, $\alpha_{1,i+n} \approx \alpha_{1,i}$, for which (20) becomes

$$\alpha_{h_{i+n}} - \alpha_{h_i} \approx L_2\left(\alpha_2^{\text{ub}} - \alpha_2\right) \quad (21)$$

which corresponds to, in a manner of speaking, a "harmonic relation" between the increment of α in subsequent iterations (due to the rest of the robot trajectory) and the quantity $(\alpha_2^{\text{ub}} - \alpha_2)$, which depends only on the obstacle characteristics. Even if this harmonic relation seems very improbable (we never observed it in experiments), it is possible in principle and needs to be taken into consideration.

VI. Subsuming Navigation Graph

Experimental results will show that μNav efficiently solves navigation problems in a time which—in practice—is

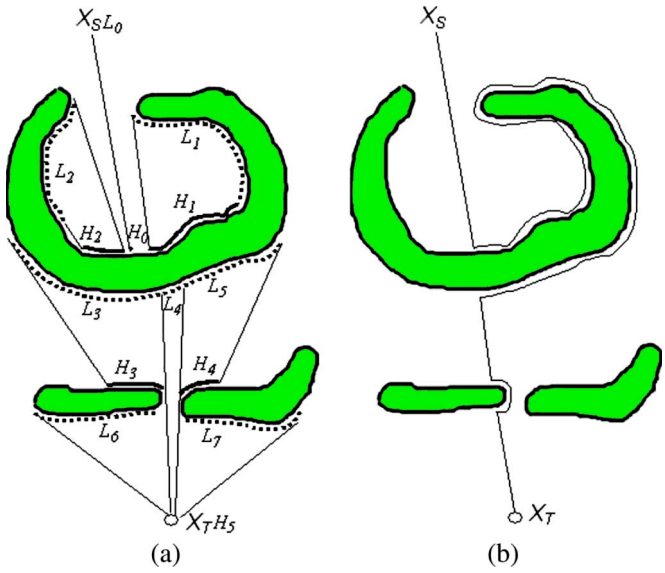

Fig. 11. (a) μNav's hit and leave regions. (b) Bug2's path.

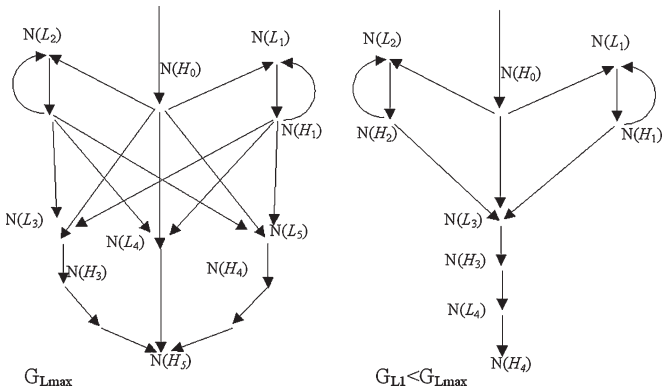

Fig. 12. Subsuming graph for (a) $U_L \approx U_{L\max}$ and (b) $U_L \ll U_{L\max}$.

comparable with other navigation algorithms. In this section, we show that μNav is complete in a probabilistic sense, i.e., in the worst case, its probability to find a path to the goal tends to one in an infinite time.

To this purpose, an abstract graphlike data structure $\mathbb{G}_L = \mathbb{G}(U_L)$ is introduced, which is a navigation graph ideally subsuming each exploration trajectory. $\mathbb{G}_L$ is only used to explain the behavior of the system; however, *neither* is it actually built *nor* searched. Its topology is implicitly defined based on the parameter U_L, where, as long as U_L increases, new areas of the workspace $\mathbb{W}$ are assimilated by $\mathbb{G}_L$, thus becoming available for the robot to explore (Figs. 11 and 12).

The law of motion in (4) and (9) generate the paths $\mathbb{P} \in \mathbb{G}_L$, each one corresponding to a set of trajectories $\mathbb{T} = \{\mathbb{T}_k\}$, such that if a trajectory $\mathbb{T}_{\text{goal}} \in \mathbb{T}$ leads from $\mathbf{x_S}$ to $\mathbf{x_T}$, every trajectory in $\mathbb{T}$ is still a solution. This concept is expressed by saying that $\mathbb{P}$ subsumes a set $\mathbb{T}$. In theory, given the configuration of obstacles, a value for U_L, $\mathbf{x_S}$, and $\mathbf{x_T}$, a map of all the candidate leave points can be built and, hence, the corresponding $\mathbb{G}_L$. Unfortunately, this cannot be achieved on the basis of pure geometric considerations. However, by considering cond_3 in Rule 2), it is easier to identify candidate *leave regions* $L(l_{iL}, l_{iR})$, i.e., intervals $l_{iL} \rightarrow l_{iR}$ on the equipotential line from which the robot is allowed to head toward the goal. Next, for each leave region L, the union of all the straight lines starting from points in L to $\mathbf{x_T}$ is considered, and a *hit region* $H(h_{iL}, h_{iR})$ is defined as their first intersection with obstacles. If a line starting in L exists, which does not intersect any obstacle, then $\mathbf{x_T} \in H(h_{iL}, h_{iR})$. Considering that $H(h_{iL}, h_{iR})$ is only guaranteed to be piecewise continuous, it is divided—if required—into smaller *hit regions* H_m; the corresponding leave region L is divided into smaller *leave regions* L_m accordingly.

A graph $\mathbb{G}_L$ is ideally built, composed of leave and hit regions with the following characteristics: 1) Each node is labeled as *leave* or *hit*, and it stores information about the ending points of the corresponding region (i.e., $l_{iL} \rightarrow l_{iR}$ for leave regions and $h_{iL} \rightarrow h_{iR}$ for hit regions), and 2) each node is connected to its successors by oriented edges. Moreover, the start node $\mathbb{N}_{\text{start}}(l_{0L} = l_{0R} = \mathbf{x_S})$ is labeled *leave*, whereas the target node $\mathbb{N}_{\text{goal}}(h_{nL} = h_{nR} = \mathbf{x_T})$ is labeled *hit*. As an example, consider Figs. 11(a) and 12, where both graphs in Fig. 12 correspond to the environment in Fig. 11(a) but the graph on the left is built when $U_L \approx U_{L\max}$, and hence, the equipotential lines can be approximated with the obstacle profile. On the contrary, the graph on the right is built when $U_L \ll U_{L\max}$; this reduces the number of leave and hit regions (which can be a good idea in an initial phase) but can prevent the robot from finding a path to the goal through narrow passages.

Considering that, by construction, there is a correspondence between a virtual exploration strategy over $\mathbb{G}_L$ and the behavior exhibited by μNav, μNav is characterized by two interesting properties.

1) The trajectories generated by μNav are robust in the presence of measurement noise.
2) The paths in $\mathbb{G}_L$ generated by μNav subsume every trajectory generated by Bug2, where completeness is guaranteed.

With respect to statement 1), note that a μNav trajectory to the goal $\mathbb{T}_{\text{goal}}$ allows for significant *deviations* while still remaining within the same set $\mathbb{T}$ (i.e., within the same path $\mathbb{P}$). As long as the robot trajectory lies within $\mathbb{T}$, localization issues can be safely ignored. This can also be said as follows. If $\mathbb{N}_{\text{goal}} \in \mathbb{G}$ is a successor of a leave node $\mathbb{N}(l_{iL}, l_{iR})$, then $\mathbf{x_T}$ can be reached from *every* point in $l_{iL} \rightarrow l_{iR}$ moving along a straight line. If a hit node $\mathbb{N}(h_{iL}, h_{iR})$ is a successor of the leave node $\mathbb{N}(l_{iL}, l_{iR})$ and a path to $\mathbf{x_T}$ exists, which follows a straight line between the leave point $l_i \in l_{iL} \rightarrow l_{iR}$ and the hit point $h_i \in h_{iL} \rightarrow h_{iR}$, the actual trajectory is still a path to $\mathbf{x_T}$ *whichever* point $l'_i \in l_{iL} \rightarrow l_{iR}$ the robot chooses to leave the boundary. In fact, the new hit point h'_i is still composed of $h_{iL} \rightarrow h_{iR}$, thus not affecting the remaining trajectory.

With respect to statement 2), note that Bug2 segments of trajectory $h_i \rightarrow l_{i+1}$ almost overlap with the μNav potential arcs, whereas Bug2 segments $l_i \rightarrow h_i$ are straight lines toward the goal. Therefore, a solution generated by Bug2 is necessarily subsumed by a path $\mathbb{P}$ and corresponds to a set of trajectories $\mathbb{T}$, as Bug2's hit and leave points lie on a corresponding μNav hit or leave region. Considering that Bug2 is complete, μNav's completeness only depends on the ability of μNav rules to generate one of the trajectories in $\mathbb{T}$. Finally, because $\mathbb{G}_L$

depends on U_L, it is guaranteed to subsume every possible Bug2 trajectory only when $U_L = U_{L\max}$.

These statements have important consequences; suppose, for example, that a trajectory $\mathbb{T}_{\text{goal}}$ to the goal exists, and, hence, a path $\mathbb{P}_{\text{goal}}$ in $\mathbb{G}$. Considering that $\mathbb{G}$ contains a finite set of leave nodes (each corresponding to a leave region), finding $\mathbb{P}_{\text{goal}}$ (and, hence, $\mathbb{T}_{\text{goal}}$) requires one to choose the right sequence of leave nodes (i.e., the right sequence of leave regions) where it is convenient to head to the goal. Suppose that this can be done on a probabilistic basis and that all leave regions L belonging to the same obstacle have a probability $P(L) > 0$ to be chosen, with $\sum_{L\in\text{obstacle}} P(L) = 1$. The probability $P(\mathbb{P})$ of a particular path is given by the product of the probabilities of each choice which generated that path, i.e.,

$$P(\mathbb{P}) = \prod_{L\in\mathbb{P}} P(L). \tag{22}$$

By defining $\{\mathbb{P}_{\text{goal}}\}$ as the set of all paths of finite length which reach the goal and $\{\mathbb{P}_{\neg\text{goal}}\}$ as the set of all paths of infinite length which do not reach the goal, the probability that the robot has of taking a path that *never reaches* the goal is

$$P(\neg\text{goal}) = \sum_{\mathbb{P}\in\{\mathbb{P}_{\neg\text{goal}}\}} P(\mathbb{P}) \tag{23}$$

whereas the probability of reaching the goal (when $t \to \infty$) obviously tends to

$$P(\text{goal}) = \sum_{\mathbb{P}\in\{\mathbb{P}_{\text{goal}}\}} P(\mathbb{P}) = 1 - P(\neg\text{goal}). \tag{24}$$

Considering that all paths $\mathbb{P} \in \{\mathbb{P}_{\neg\text{goal}}\}$ have an infinite length by definition (by assuming that the robot stops only when it reaches the goal), their probability $P(\mathbb{P}) \to 0$ as $t \to \infty$ because they are computed as the product of infinite terms $P(L)$, with $0 < P(L) < 1$. Consequently, $P(\text{goal}) = 1 - P(\neg\text{goal}) \to 1$ as $t \to \infty$.

This intuitive result allows for the introduction of the following final theorem.

Theorem 11: Given that a path to the goal exists for $U_{L\max}$ and that K_2 in Line 16 of Algorithm 1 is a randomly chosen real positive number, μNav is complete in a probabilistic sense, i.e., the probability of finding a path to the goal tends to one as $t \to \infty$.

Proof: Theorem 10 shows that Line 16 of Algorithm 1 is "periodically" executed when the robot is trapped in a loop, which determines the random quantity $K_2 > 0$ to be added to α when μNav switches to *heading to the goal*. This is sufficient to state that for each obstacle, there exists a time t_1 such that $\forall t > t_1$, each leave region L belonging to that obstacle has a nonnull probability that cond_3 becomes true when the robot is in L (possibly after having looped around the obstacle many times). When $t \to \infty$, all leave regions have a nonnull probability to be chosen, and hence, the probability to find a path tends to one. ■

From Theorem 11, μNav could appear to be not very efficient. However, it should be noticed that the random quantity K_2 is extremely small when compared with other contributions (in practice, we set $K_2 \approx 0$). μNav manages to find a path to the goal, thanks to its intrinsic properties, whereas K_2 has the main purpose of theoretically guaranteeing completeness (in a probabilistic sense) even in very unlucky situations, such as when there is an "almost harmonic relation," as depicted in (18)–(21). If required, (21) can provide cues to choose a reasonable interval of values for randomly choosing K_2. In fact, it seems convenient to assign K_2 a uniform probability in the interval $[0 \quad K_{\text{gain}}(\Delta\alpha_i^{\text{ub}} - \Delta\alpha_i)]$, where $\Delta\alpha_i$ and $\Delta\alpha_i^{\text{ub}}$ correspond, respectively, to the increment of α and α^{ub}, which have been measured during the last *obstacle segment* O_i. Note, in fact, that $(\Delta\alpha_i^{\text{ub}} - \Delta\alpha_i) \approx L_n(\alpha_2^{\text{ub}} - \alpha_2)$ as the number of loops L_n performed around O_i increases, which allows one to establish a relationship between K_2 and the characteristics of the obstacles which are present in the environment. If this is the case, $K_{\text{gain}} \ll 1$ can be arbitrarily chosen to allow for a, more or less, quick transition toward a purely probabilistic algorithm. In addition, in most situations, the number of leave regions is small, considering that μNav allows for significant deviations from the trajectory while still remaining within the same path $\mathbb{P}$. Therefore, even in the worst cases where the exploration of the subsuming graph is performed "almost" randomly, μNav is more efficient than an algorithm which performs random navigation in the Cartesian space.

Finally, note that, if required, comparing the behavior of α, β, and α^{ub} can provide heuristics to define termination conditions. In fact, the only situation for which a path cannot be found (possibly in an infinite time) is when $U_{L\max}$ divides the workspace in two nonconnected components, with one containing the robot and the other the goal. In this case, α does not increase, whereas β and α^{ub} increase every loop. If this special condition is verified for a "sufficiently long" time, one could decide to stop navigation, although this does not guarantee—formally speaking—that the goal is not reachable.

VII. Experimental Results

During the past decade, many experiments have been performed both in simulation within the MissionLab framework[1] and with different real robots, among which with an ActivMedia wheeled robot AmigoBot (Figs. 22 and 23) and the hexapod robot Sistino (Figs. 24 and 25). In this paper, simulations compare the performance of μNav and Bug2, which is the optimal algorithm (in a statistical sense) that guarantees completeness with a sensing range close to zero. In particular, the ratio $\Omega = \omega_{\mu\text{Nav}}/\omega_{\text{Bug2}}$ is computed, where the numerator and the denominator are the number of steps used by μNav and Bug2 to reach $\mathbf{x_T}$. For these experiments, the following are assumed: 1) Robot sensing and control are not affected by noise; thus, the basic algorithms are compared from the algorithmic perspective only; 2) in Bug2, it is necessary to decide *a priori* whether the robot must turn *left* or *right* when approaching an obstacle (both choices are mediated in $\Omega = \omega_{\mu\text{Nav}}/\omega_{\text{Bug2}}$). In μNav, the tangent vector $\mathbf{t}$, when approaching an obstacle, is chosen based on (5); and 3) both algorithms are implemented in such a way

[1] Available at http://www.cc.gatech.edu/ai/robot-lab/research/MissionLab.

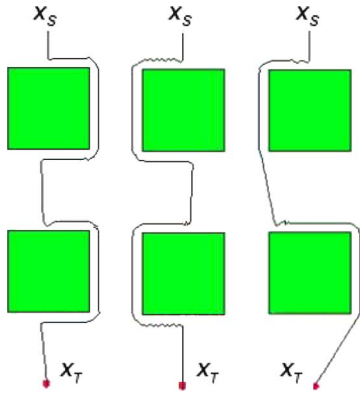

Fig. 13. Blocks. (a) and (b) Bug2. (c) μNav.

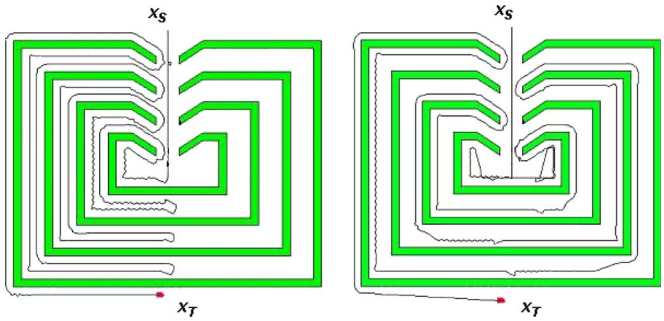

Fig. 14. Rooms. (a) Bug2. (b) μNav.

that whenever the robot can see a free path to the goal, it directly heads toward it, no matter what the algorithm says.

A. Experiments in Simulation

The following environments with an increasing degree of complexity are considered: *blocks*, *rooms*, *spirals*, *S-shaped obstacles*, *officelike environments*, and *cluttered environments*. Finally, variations of the base patterns with added *random obstacles* and *noise* are taken into account.

1) Blocks: Experiments are carried out by varying the following: 1) the number of blocks the robot has to face while navigating toward the goal; 2) their dimension; and 3) the orientation of the obstacle's axis with respect to the straight line connecting $\mathbf{x_S}$ to $\mathbf{x_T}$ (Fig. 13). By varying the number of blocks n, the mean ratio $\overline{\Omega}_n \approx 0.8$; μNav performs better than Bug2 in reaching $\mathbf{x_T}$, and this is more and more evident as soon as n increases. The same happens when varying the block dimension d from *small* to *big* ($\overline{\Omega}_d \approx 0.9$) and when varying the orientation of the obstacle's axis with respect to the straight line connecting $\mathbf{x_S}$ to $\mathbf{x_T}$. This is a consequence of the fact that μNav stops following the obstacle's boundary as soon as it is possible to head toward the goal, whereas Bug2's leave points are necessarily located on the straight line connecting $\mathbf{x_S}$ to $\mathbf{x_T}$.

2) Rooms: Experiments are carried out by varying the following: 1) the number of concentric rooms n; 2) the room dimension d from *small* to *big*; and 3) the number s of *stacked* rooms (Figs. 14 and 15). By varying the number of concentric rooms (Fig. 14), the average ratio $\overline{\Omega}_n \approx 1.2$. In particular, if n is small (i.e., $n < 4$) Bug2 outperforms μNav, considering that the latter takes some time before recognizing that the robot

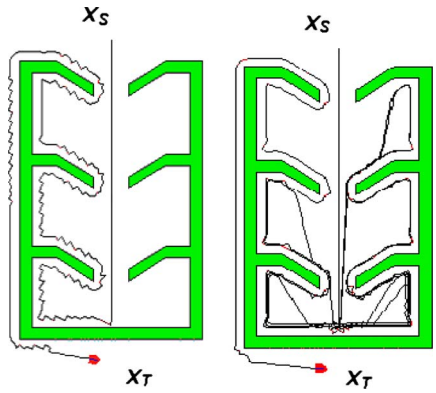

Fig. 15. Rooms. (a) Bug2. (b) μNav.

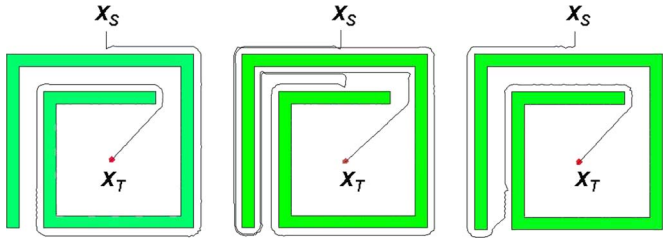

Fig. 16. Spirals. (a) and (b) Bug2. (c) μNav.

is trapped in a deadlock. However, as n increases, μNav tends to perform as good as Bug2, and $\overline{\Omega}_n \approx 1$. When varying the dimension of the room, Bug2 is always more efficient than μNav, yielding an average ratio of $\overline{\Omega}_d \approx 1.5$. Finally, when considering stacked obstacles, μNav's drawbacks are evident (Fig. 15); Bug2 computes the distance to the goal and chooses to leave the boundary of the obstacle only if such distance is smaller than the distance computed from the last hit point, thus guaranteeing that—during navigation—this distance tends to zero. Considering that μNav (in contrast with Bug2) does not assume any self-localization capabilities; it ideally explores many *leave segments* in the subsuming graph, thus requiring a longer time to find a solution. This finally yields an average ratio of $\overline{\Omega}_s \approx 2.5$.

3) Spirals: Experiments are carried out by varying the following: 1) the number of spires n, the mutual position (inside/outside the spiral) of $\mathbf{x_S}$ and $\mathbf{x_T}$, and 2) the orientation of the obstacles' axis with respect to the straight line connecting $\mathbf{x_S}$ to $\mathbf{x_T}$ (Fig. 16). By averaging over the number of spires from $n = 1$ to $n = 4$, it holds $\overline{\Omega}_n \approx 0.4$, a value that monotonically decreases as n increases. This is due to the fact that depending on the shape of the spiral itself, the performances of the *left* and *right* Bug2 are radically different. The *left* Bug2 [Fig. 16(a)] is almost comparable to μNav [Fig. 16(c)], whereas *right* Bug2 [Fig. 16(b)] behaves much worse. This is even more evident when the number of spires increases; considering that Bug2 and μNav deal with local information, one is not allowed to decide in advance whether to adopt a *left* or *right* strategy for Bug2. On the contrary, μNav does not have similar problems, considering that the tangent is chosen in runtime. Similar results are still valid when moving $\mathbf{x_S}$ and $\mathbf{x_T}$.

4) S-Shaped Obstacles: Experiments are carried out by varying the following: 1) the obstacle dimension d from *small*

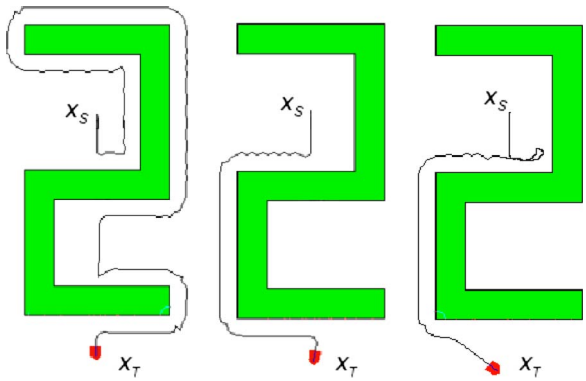

Fig. 17. S-shaped obstacles. (a) and (b) Bug2. (c) μNav.

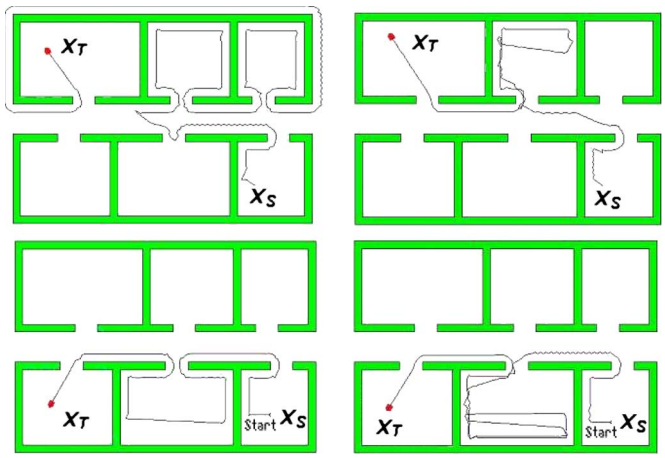

Fig. 18. Officelike environments. (a) and (b) Bug2. (c) and (d) μNav.

to *big*; 2) the position of both $\mathbf{x_S}$ and $\mathbf{x_T}$ inside or outside concavities; and 3) the orientation of the obstacle's axis (Fig. 17). Similar to spirals, the performances of the *left* and *right* Bug2 are radically different [Fig. 17(a) and (b)], whereas μNav is able to recover when the robot is heading toward a direction which turns out to be not very promising [Fig. 17(c)]. When averaging over different cases, $\overline{\Omega} \approx 0.8$.

5) Officelike Environments: More complex officelike environments are considered. In Fig. 18, it is possible to identify a central corridor, surrounded on both sides by a given number of rooms n, which varies in the experiments. The performance of Bug2 and μNav vary significantly depending on the mutual position of $\mathbf{x_S}$ and $\mathbf{x_T}$. For example, on the top portion of Fig. 18, μNav outperforms Bug2, with a ratio of $\Omega \approx 0.5$; on the bottom part of Fig. 18, the opposite is true, and $\Omega \approx 1.4$. Once again, the performances of the *left* and *right* Bug2 are radically different. In Fig. 18(a), Bug2 turns on the right, which is not very convenient because the goal could be easily reached by turning on the left. The performance of μNav, instead, depends only on the topology of the environment.

6) Cluttered Environments: Experiments with a safety distance which varies during exploration have been performed (Fig. 19). In μNav, U_L is initially set to a very low value, which forces the robot to stay far from obstacles. If a path to the goal is not found, U_L progressively increases, thus allowing one to access unexplored areas. The ratio Ω varies radically depending on the goal location; if the goal is outside

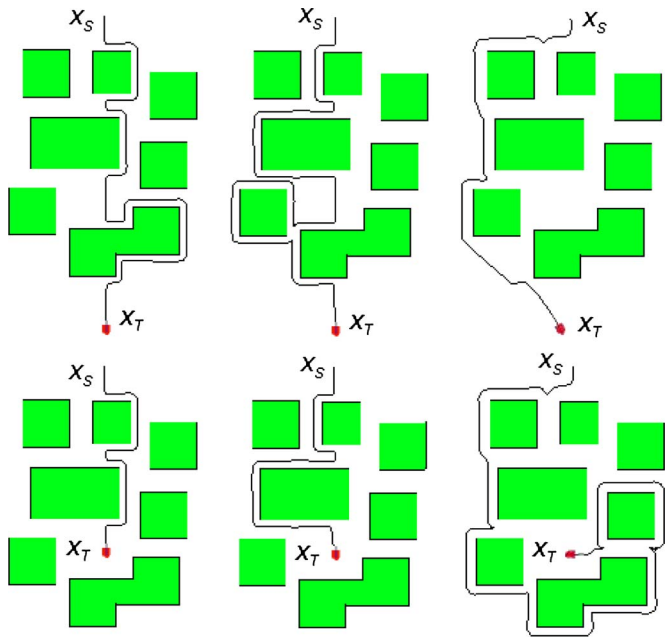

Fig. 19. Cluttered environments. (a), (b), (d), and (e) Bug2. (c) and (f) μNav.

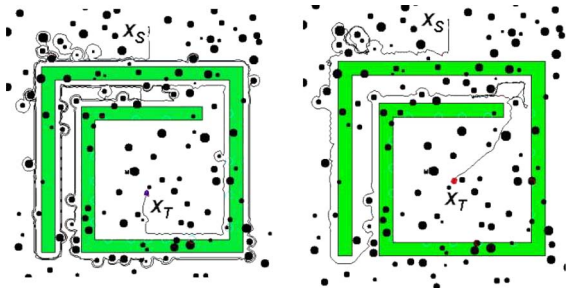

Fig. 20. Random obstacles. (a) Bug2. (b) μNav.

the cluster of obstacles (experiments on the top line), μNav is more efficient, considering that Bug2 is always required to follow the boundary of obstacles, which is revealed to be a waste of time in cluttered environments [Fig. 19(a) and (b)]. In this case, μNav deals with the clusters of obstacles as if they were a single obstacle, which turns out to be a winning strategy [Fig. 19(c)]. If, on the opposite, the goal is inside the cluster of obstacles (experiments on the bottom line), Bug2 is more efficient [Fig. 19(d) and (e)]; μNav is initially very cautious, and it is allowed to enter unexplored areas only after having performed an almost complete loop and having opportunely raised U_L [Fig. 19(f)].

7) Base Patterns and Random Obstacles: Two sets of experiments with random noise have been performed. In the first case, random obstacles are added to base patterns (Fig. 20); in the second case, realistic sensor noise is added (Fig. 21). When repeating all previous experiments by adding random obstacles, an average ratio $\overline{\Omega} \approx 1.4$ is obtained; however, this value must be taken *cum grano salis*, considering that its variance is high depending on the topology of the workspace as a whole. When adding realistic sensing errors, the average ratio $\overline{\Omega}$ decreases as noise increases. Fig. 21(a) shows a case that depicts Bug2 in the absence of noise. Fig. 21(b) shows a case where errors seriously affect the "length" quality factor of Bug2. Fig. 21(c)

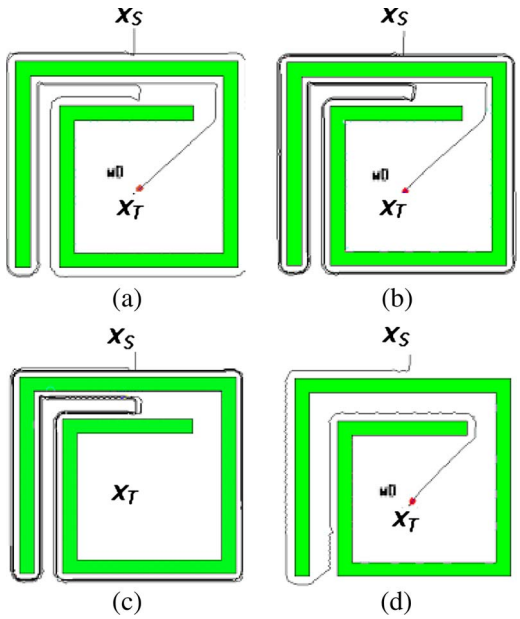

Fig. 21. Sensing noise. (a), (b), and (c) Bug2. (d) μNav.

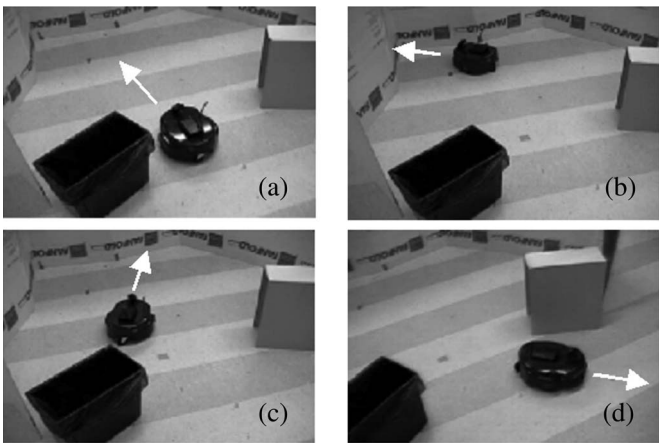

Fig. 22. Experiments with AmigoBot.

shows a case where a certain level of noise can even make Bug2 loop indefinitely, considering that the discrete choice of the next leave point requires severe precision and repeatability. In μNav [case shown in Fig. 21(d)], exploration is much less critical.

B. Experiments With Real Robots

In order to validate μNav in real scenarios, many experiments have been performed using an ActivMedia AmigoBot and the home-built hexapod Sistino.

1) Experiments With AmigoBot: Considering that the AmigoBot is specifically designed for educational purposes, it is equipped with very poor sensors; we rely only on the onboard ultrasonic sensors for obstacle detection, thus disregarding the onboard camera. Moreover, in this set of experiments, onboard odometry is used (which is particularly unreliable in the AmigoBot) only for the purpose of indirectly computing the direction of the target $\mathbf{x_T}$. Fig. 22 shows the robot escaping from a $\mathbb{T}_1$ deadlock. When "realizing" that it is looping within a closed area, μNav forces the robot to follow the wall [Fig. 22(d)] instead of heading toward $\mathbf{x_T}$ [Fig. 22(c)]. Fig. 23 shows an

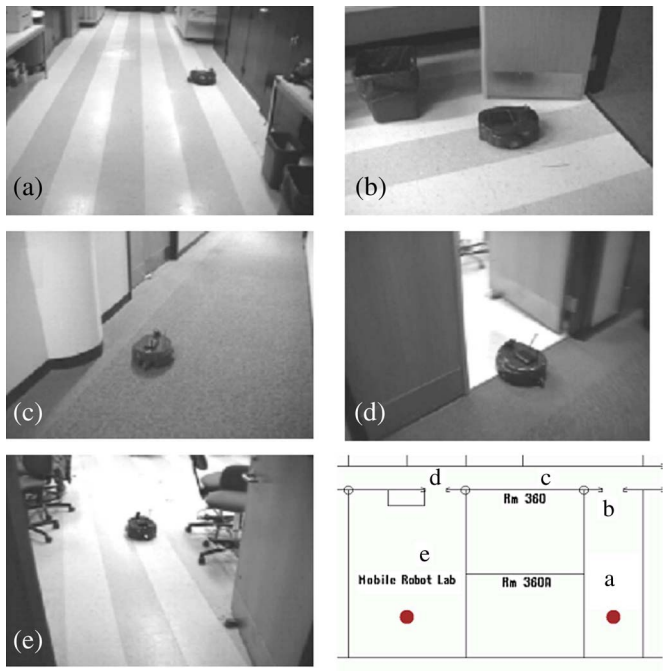

Fig. 23. AmigoBot navigating in a structured environment.

experiment performed in a structured environment. The robot is required to move from one room to another, without either a map or a set of via points. However, it manages to exit from the first room and then to find a path to $\mathbf{x_T}$.

Finally, tests aimed at stressing the robustness of μNav to noise have been performed, e.g., by purposively hitting the robot, thus suddenly changing its direction of motion. Considering that μNav does not rely on localization to explore the environment, the robot manages to find its way out from mazes despite big and unforeseen errors in orientation.

2) Experiments With Sistino: The onboard sensory of the Sistino hexapod is even poorer than that of the AmigoBot. Considering that the robot does not perform odometric reconstructions at all, the azimuth direction to the goal is given by photodiodes which are used to detect a light source at a maximum distance of 6 m with a precision of about 5°; the distance to the closest obstacle is given by a frontal rotating sonar that is able to scan for obstacles within a 180° cone and by six infrared proximity sensors around the robot's body. During navigation, Sistino heads toward the light (i.e., $\mathbf{x_T}$), always finding its way out of different mazes with a complexity (Figs. 24 and 25) which is comparable to the one of simulations.

VIII. Conclusion

In this paper, a new framework for mobile robot navigation, called μNav, has been theoretically presented and experimentally evaluated. Specifically, the key ideas underlying its design are as follows.

Minimal information. Based on the classification introduced in Section II, μNav is able to solve the navigation problem using just only two (in case of an "ideal" touch sensor) or three *information units*. This is a rather different approach with respect to current research guidelines about competitive strategies.

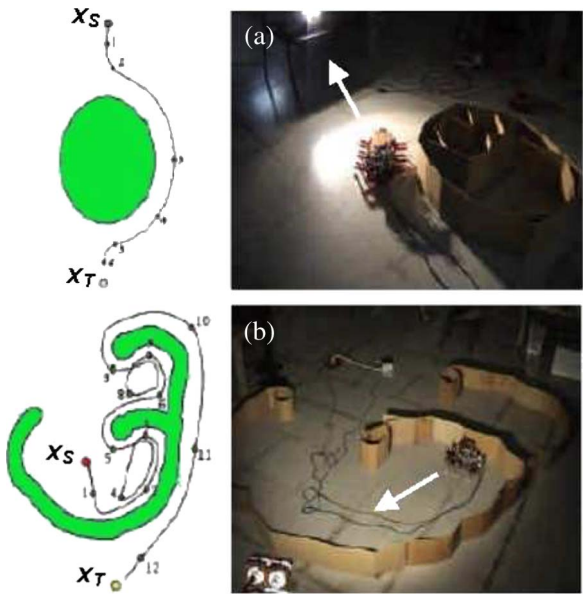

Fig. 24. Experiments with the hexapod robot Sistino.

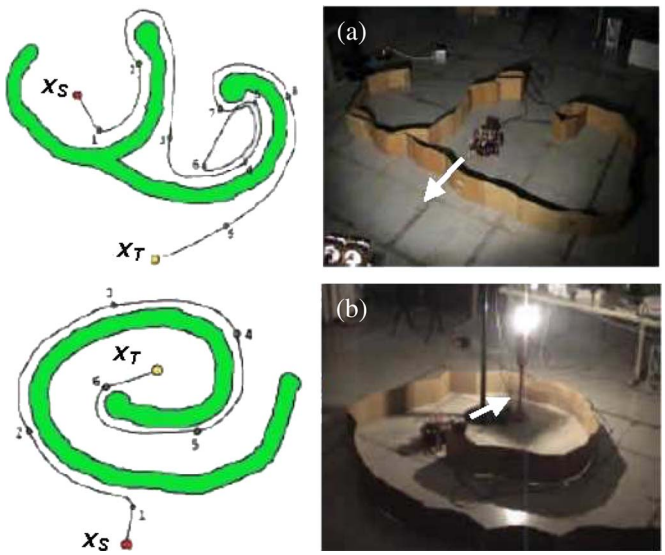

Fig. 25. Experiments with the hexapod robot Sistino.

Motion law. The motion law described in Section III allows for a continuous sensor-based approach to path planning. Furthermore, such a law is at the basis of a principled framework for deadlock detection.

Lack of self-localization. In contrast with what is assumed in other algorithms (Section II), μNav does not rely on any form of Cartesian self-localization. This is a very desirable feature, considering that self-localization is rather difficult in many scenarios. For instance, it is possible to foresee the adoption of μNav in a broad spectrum of robotics applications, among others, humanoids—for which self-localization is still subject to ongoing research—and planetary exploration rovers—which usually exploit *sun sensors*, i.e., sensory information very similar to the azimuth direction.

However, as in other minimalist approaches to navigation, issues related to *completeness* and *robustness* have been addressed. With respect to *completeness*, two observations must be done. The first is that μNav is *complete* in a probabilistic sense. Theoretically, this means that the probability of finding a solution to the navigation problem tends to be certain in an infinite amount of time (Fig. 26). This is very common for algorithms presented in the past few years, e.g., sample-based motion planners [6]; it is well known that in most cases, they are comparable with deterministic planners. This behavior has been observed also in our experiments; μNav is always able to find its way to the goal and, in significant cases, it originates shorter paths with respect to Bug2. This is a surprising result, given that the *information space* of μNav is more limited. In fact, it is more limited than the *information space* of any other navigation algorithm, Pledge apart. The second observation is that reasonable *termination conditions* can be easily defined—although through heuristics—by inspecting the behavior of internal parameters.

Fig. 26. Simulated navigation in the map of Venice.

With respect to *robustness*, two considerations can be done. The first is related to the *workspace*. As it has been described in Section VI, considering that the μNav exploration strategy is not based on actual geometrical configurations of obstacles, it can deal with both static and moving obstacles. This is particularly interesting when considering algorithms with a greater *information space*, which nevertheless assume either static obstacles or ideal sensors to determine the obstacle type. The second refers to *sensing* and *control noise*; as a matter of fact, μNav proves to be very robust. Experimental evaluation (Section VII) validates this claim. Sistino adopts very noisy sensors to perform navigation toward the light source, whereas AmigoBot is able to find the goal location even when using raw odometry to simulate knowledge of goal direction. On the contrary, other minimalist algorithms assume ideal or unrealistic sensing and motion capabilities to such an extent that several works [34], [57], [63] specifically investigate their behavior when inflating noise. From these promising results, μNav is currently being applied to planetary and multirobot explorations.

REFERENCES

[1] D. Shi, E. Collins, and D. Dunlap, "Robot navigation in cluttered 3-D environments using preference-based fuzzy behaviors," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 37, no. 6, pp. 1486–1500, Dec. 2007.

[2] K. Kutulakos, V. Lumelsky, and C. Dyer, "Vision-guided exploration: A step toward general motion planning in three dimensions," in *Proc. IEEE ICRA*, Atlanta, GA, 1993, pp. 289–296.

[3] I. Kamon, E. Rimon, and E. Rivlin, "Range-sensor based navigation in three dimension," in *Proc. IEEE ICRA*, Detroit, MI, 1999, pp. 163–169.

[4] Z. Sun and J. Reif, "On robotic optimal path planning in polygonal regions with pseudo-Euclidean metrics," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 37, no. 4, pp. 925–937, Aug. 2007.

[5] H. Choset, K. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. Kavraki, and S. Thrun, *Principles of Robot Motion*. Cambridge, MA: MIT Press, 2005.

[6] S. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[7] S. Ge, X. Lai, and A. A. Mamun, "Boundary following and globally convergent path planning using instant goals," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 35, no. 2, pp. 240–255, Apr. 2005.

[8] X. Yang, M. Moallem, and R. Patel, "A layered goal-oriented fuzzy motion planning strategy for mobile robot navigation," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 35, no. 6, pp. 1214–1225, Dec. 2005.

[9] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[10] O. Trullier, S. Wiener, A. Berthoz, and J. Meyer, "Biologically-based artificial navigation systems: Review and prospects," *Prog. Neurobiol.*, vol. 51, no. 5, pp. 483–544, 1997.

[11] M. Franz and H. Mallot, "Biomimetic robot navigation," *Robot. Auton. Syst.*, vol. 30, no. 1, pp. 133–153, Jan. 2000.

[12] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[13] R. Brooks, "A robust layered control system for a mobile robot," *IEEE J. Robot. Autom.*, vol. RA-2, no. 1, pp. 14–23, Mar. 1986.

[14] R. Arkin, "Motor schema—Based mobile robot navigation," *Int. J. Robot. Res.*, vol. 8, no. 4, pp. 93–112, 1989.

[15] B. Kuipers and Y. Byun, "A robot exploration and mapping strategy based on a semantic hierarchy of spatial representation," *Robot. Auton. Syst.*, vol. 8, pp. 47–63, 1991.

[16] M. Mataric, "Integration of representation into goal-driven behavior-based robots," *IEEE Trans. Robot. Autom.*, vol. 8, no. 3, pp. 304–312, Jun. 1992.

[17] M. Slack, "Navigation templates: Mediating qualitative guidance and quantitative control in mobile robots," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 2, pp. 452–466, Mar./Apr. 1993.

[18] J. Donnart and J. Meyer, "Hierarchical map building and self-positioning with MonaLysa," *Adapt. Behav.*, vol. 5, no. 1, pp. 29–74, 1996.

[19] R. Pfeifer and C. Scheier, *Understanding Intelligence*. Boston, MA: MIT Press, 1999.

[20] I. Kamon and E. Rivlin, "Sensory-based motion planning with global proofs," *IEEE Trans. Robot. Autom.*, vol. 13, no. 6, pp. 814–822, Dec. 1997.

[21] J. O'Kane and S. LaValle, "On comparing the power of mobile robots," in *Proc. Robot.: Sci. Syst. II*, Philadelphia, PA, Aug. 16–19, 2006.

[22] B. Tovar, R. Murrieta-Cid, and S. LaValle, "Distance-optimal navigation in an unknown environment without sensing distances," *IEEE Trans. Robot. Autom.*, vol. 23, no. 3, pp. 506–518, Jun. 2007.

[23] Y. Koren and J. Borenstein, "Potential field methods and their inherent limitations for mobile robot navigation," in *Proc. IEEE ICRA*, Sacramento, CA, Apr. 1991, pp. 1398–1404.

[24] D. Sleator and R. Tarjan, "Amortized efficiency of list update rules," in *Proc. 17th Annu. ACM Symp. Theory Comput.*, 1984, pp. 202–208.

[25] R. Karp, "On-line algorithms versus off-line algorithms: How much is it worth to know the future?" in *Proc. IFIP 12th World Comput. Congr. Algorithms, Softw., Architecture Inf. Process.,* 1992, vol. 12, pp. 416–429.

[26] H. Everett, *Sensors for Mobile Robots: Theory and Application*. Wellesley, MA: A.K. Peters, Ltd., 1995.

[27] B. Donald, "On information invariants in robotics," *Artif. Intell.*, vol. 72, no. 1/2, pp. 217–304, Jan. 1995.

[28] M. Erdmann, "Understanding actions and sensing by designing action-based sensors," *Int. J. Rob. Res.*, vol. 14, no. 5, pp. 483–509, Oct. 1995.

[29] C. Papadimitriou and M. Yannakakis, "Shortest paths without a map," in Automata, Languages and Programming. Berlin, Germany: Springer-Verlag, 1989, vol. 372.

[30] H. Noborio, Y. Haeda, and K. Urakawa, "A comparative study of sensor-based path-planning algorithms in an unknown maze," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Yakamatsu, Japan, Oct. 2000, pp. 909–916.

[31] C. Icking, T. Kamphans, R. Klein, and E. Langetepe, "On the competitive complexity of navigation tasks," in Sensor Based Intelligent Robots. Berlin, Germany: Springer-Verlag, 2002, vol. 2238, pp. 245–258.

[32] A. Datta and S. Soundaralakshmi, "On-line path planning in an unknown polygonal environment," *Inf. Sci.*, vol. 164, no. 1–4, pp. 89–111, Aug. 2004.

[33] J. Ng and T. Braunl, "Performance comparison of bug navigation algorithms," *J. Intell. Robot. Syst.*, vol. 50, no. 1, pp. 73–84, Sep. 2007.

[34] J. O'Kane and S. LaValle, "Localization with limited sensing," *IEEE Trans. Robot.*, vol. 23, no. 4, pp. 704–716, Aug. 2007.

[35] J. O'Kane and S. LaValle, "Sloppy motors, flaky sensors, and virtual dirt: Comparing imperfect ill-informed robots," in *Proc. IEEE ICRA*, Rome, Italy, Apr. 2007, pp. 4084–4089.

[36] J. O'Kane and S. LaValle, "Comparing the power of robots," *Int. J. Robot. Res.*, vol. 27, no. 1, pp. 5–23, Jan. 2008.

[37] V. Lumelsky and A. Stepanov, "Dynamic path planning for a mobile automation with limited information on the environment," *IEEE Trans. Autom. Control*, vol. AC-31, no. 11, pp. 1058–1063, Nov. 1986.

[38] V. Lumelsky and A. Stepanov, "Path planning strategies for point mobile automation moving amidst unknown obstacles of arbitrary shape," *Algorithmica*, vol. 2, pp. 403–430, 1987.

[39] H. Noborio, "A path planning algorithm for generation of an intuitively reasonable path in an uncertain 2D workspace," in *Proc. Jpn.-USA Symp. Flexible Autom.*, Kyoto, Japan, 1990, pp. 477–480.

[40] H. Noborio, "A sufficient condition for designing a family of sensor based deadlock free planning algorithms," *Adv. Robot.*, vol. 7, no. 5, pp. 413–433, Oct. 1993.

[41] S. Sarid, A. Shapiro, and Y. Gabriely, "MRBUG: A competitive multi-robot path finding algorithm," in *Proc. IEEE ICRA*, Rome, Italy, Apr. 2007, pp. 877–882.

[42] I. Kamon, E. Rivlin, and E. Rimon, "A new range-sensor based globally convergent navigation algorithm for mobile robots," in *Proc. IEEE ICRA*, Minneapolis, MN, Apr. 1996, pp. 429–435.

[43] I. Kamon, E. Rimon, and E. Rivlin, "TangentBug: A range-sensor-based navigation algorithm," *Int. J. Robot. Res.*, vol. 17, no. 9, pp. 934–953, 1998.

[44] V. Lumelsky and T. Skewis, "A paradigm for incorporating vision in the robot navigation function," in *Proc. IEEE ICRA*, Philadelphia, PA, 1988, pp. 734–739.

[45] V. Lumelsky and T. Skewis, "Incorporating range sensing in the robot navigation function," *IEEE Trans. Syst., Man Cybern.*, vol. 20, no. 5, pp. 1058–1068, Sep./Oct. 1990.

[46] S. Kim, J. Russel, and K. Koo, "Construction robot path-planning for earthwork operations," *J. Comput. Civil Eng.*, vol. 17, no. 2, pp. 97–104, Apr. 2003.

[47] S. Rajko and S. LaValle, "A pursuit-evasion BUG algorithm," in *Proc. IEEE ICRA*, May 2001, pp. 1954–1960.

[48] S. Sachs, S. LaValle, and S. Rajko, "Visibility-based pursuit-evasion in an unknown planar environment," *Int. J. Robot. Res.*, vol. 23, no. 1, pp. 3–26, Jan. 2004.

[49] M. Weir, A. Buck, and J. Lewis, "POTBUG: A mind's eye approach to providing BUG-like guarantees for adaptive obstacle navigation using dynamic potential fields," in *Proc. 9th Int. Conf. Simul. Adaptive Behaviour*, Roma, Italy, Sep. 2006, pp. 239–250.

[50] S. Laubach, "Theory and experiments in autonomous sensor-based motion planning with applications for flight planetary microrovers," Ph.D. dissertation, California Inst. Technol., Pasadena, CA, 1999.

[51] S. Laubach and J. Burdick, "An autonomous sensor-based path planner for planetary microrovers," in *Proc. IEEE ICRA*, Detroit, MI, May 1999, pp. 347–354.

[52] E. Magid and E. Rivlin, "CAUTIOUSBUG: A competitive algorithm for sensory-based robot navigation," in *Proc. IEEE/RSJ Int. Conf. IROS*, Sendai, Japan, Oct. 2004, pp. 2757–2762.

[53] M. Schwager, F. Bullo, D. Skelly, and D. Rus, "A ladybug exploration strategy for distributed adaptive coverage control," in *Proc. IEEE ICRA*, Pasadena, CA, May 2008, pp. 2346–2353.

[54] M. Blum and D. Kozen, "On the power of the compass (or, why mazes are easier to search than graphs)," in *Proc. 19th Symp. Found. Comput. Sci.*, Ann Arbor, MI, 1978, pp. 132–142.

[55] H. Abelson and A. diSessa, *Turtle Geometry*. Cambridge, MA: MIT Press, 1980.

[56] A. Hemmerling, *Labyrinth Problems. Labyrinth-Searching Abilities of Automata*. Leipzig, Germany: B.G. Teubner, 1989.

[57] T. Kamphans and E. Langetepe, "The Pledge algorithm reconsidered under errors in sensors and motion," in *Proc. 1st Workshop Approximation Online Algorithms*. New York: Springer-Verlag, 2003, vol. 2909.

[58] V. Lumelsky and S. Tiwari, "An algorithm for maze searching with azimuth input," in *Proc. IEEE ICRA*, San Diego, CA, May 1994, pp. 111–116.

[59] A. Sankaranarayanan and M. Vidasagar, "A new path planning algorithm for moving a point object amidst unknown obstacles in a plane," in *Proc. IEEE ICRA*, Cincinnati, OH, May 15–19, 1990, pp. 1930–1936.
[60] A. Sankaranarayanan and M. Vidasagar, "Path planning for a moving point object amidst unknown obstacles in a plane: A new algorithm and a general theory for algorithm development," in *Proc. IEEE Int. CDC*, Dec. 1990, pp. 1111–1119.
[61] H. Noborio and K. Urakawa, "Three or more dimensional sensor-based path-planning algorithm HD-I," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Taejon, South Korea, Oct. 1999, pp. 1699–1706.
[62] H. Noborio, R. Nagami, and S. Hirao, "A new sensor-based path-planning algorithm whose path length is shorter on the average," in *Proc. IEEE ICRA*, New Orleans, LA, 2004, pp. 2832–2839.
[63] B. Donald, "The complexity of planar compliant motion planning with uncertainty," *Algorithmica*, vol. 5, no. 3, pp. 353–382, 1990.

Fulvio Mastrogiovanni (S'05) received the M.S. degree in computer science and the Ph.D. degree in robotics from the University of Genova, Genova, Italy.

He is currently a Research Assistant with the Department of Communication, Computer, and System Sciences, University of Genova, where he works on the integration between autonomous robotics and ambient intelligence, with a special focus on sensor fusion and knowledge representation, as well as mobile robot navigation and self-localization.

Antonio Sgorbissa (M'05) received the M.S. degree in electronic engineering and the Ph.D. degree in robotics from the University of Genova, Genova, Italy.

Currently, he is an Assistant Professor in computer science with the Faculty of Engineering and of the Department of Humanities, University of Genova, and he leads the Mobile Robotics "Laboratorium," Department of Communication, Computer, and System Sciences. His main research interests are intelligent autonomous systems, with a special focus on indoor and outdoor mobile robotics, distributed multirobot systems, planning, navigation, and control.

Renato Zaccaria (M'05) received the M.S. degree in electrical engineering from the University of Genova, Genova, Italy.

He started working on robotics in the '70s in one of the earlier Italian research groups, which is characterized by a strong interdisciplinary approach. He is currently a Full Professor in computer science with the Department of Communication, Computer, and System Sciences, University of Genova and is the local Coordinator of the European Master on Advanced Robotics. In 2000, he founded one of the Department's spin-off companies "Genova Robot," whose present R&D activity is strongly linked to academic research. His current research interest mainly includes service robotics.

Mr. Zaccaria has been awarded the title Commendatore della Repubblica because of his activity in technology transfer in robotics.